

R5.07

Automatisation de la chaîne de production

Jean-Yves Colin
IUT du Havre
Département Informatique
v1.02- 2023-2024

Stratégie de branchement de GitHub



Stratégie de branchement de GitHub



- Basé sur une unique branche principale permanente ('main'), et des branches temporaires de cette branche principale ('feature branches').
 - chaque branche temporaire correspond à une demande de nouvelle fonctionnalité, de correction ou d'amélioration

Stratégie de branchement de GitHub



branche principale



Stratégie de branchement de GitHub

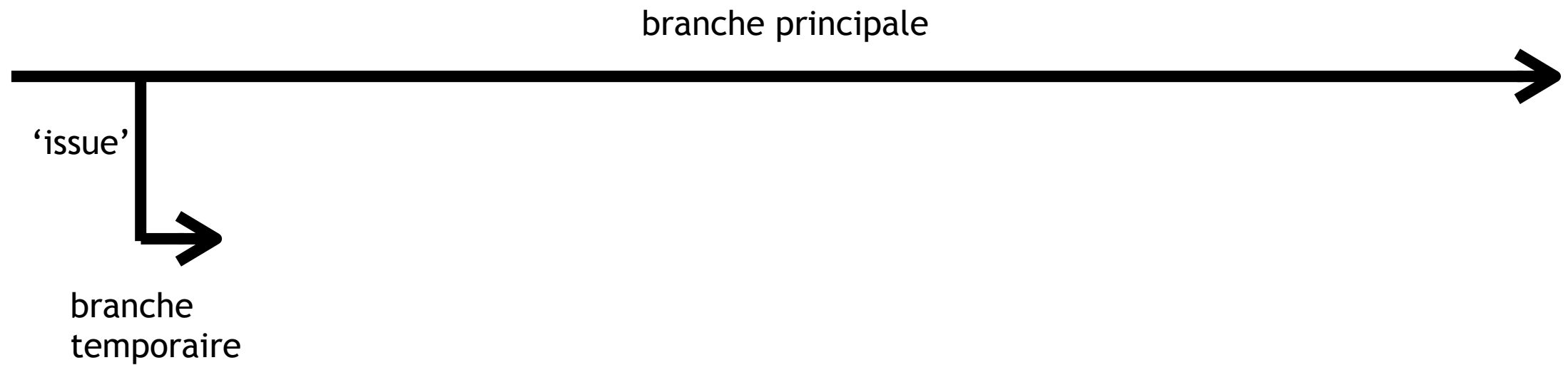


branche principale

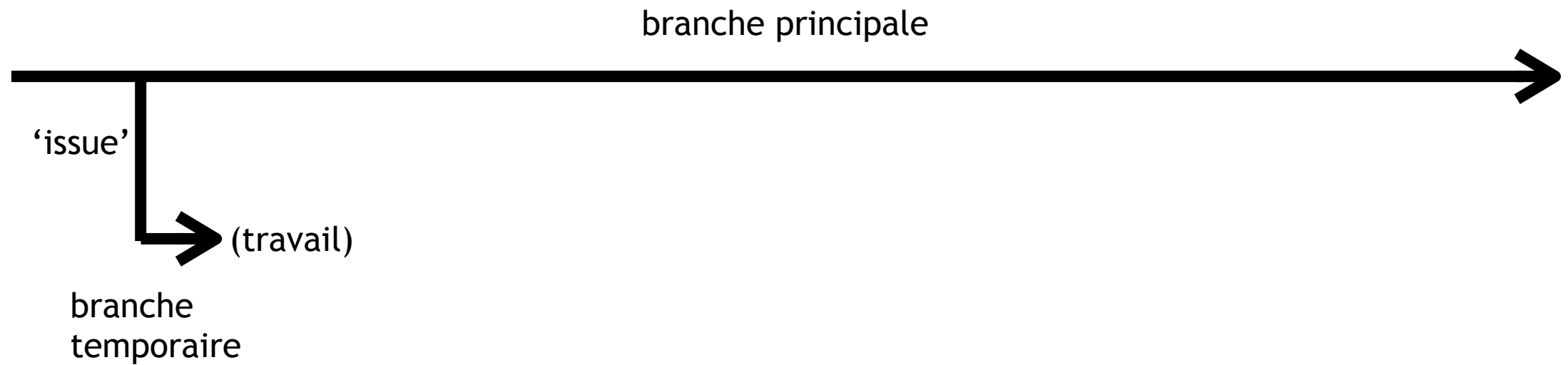


‘issue’

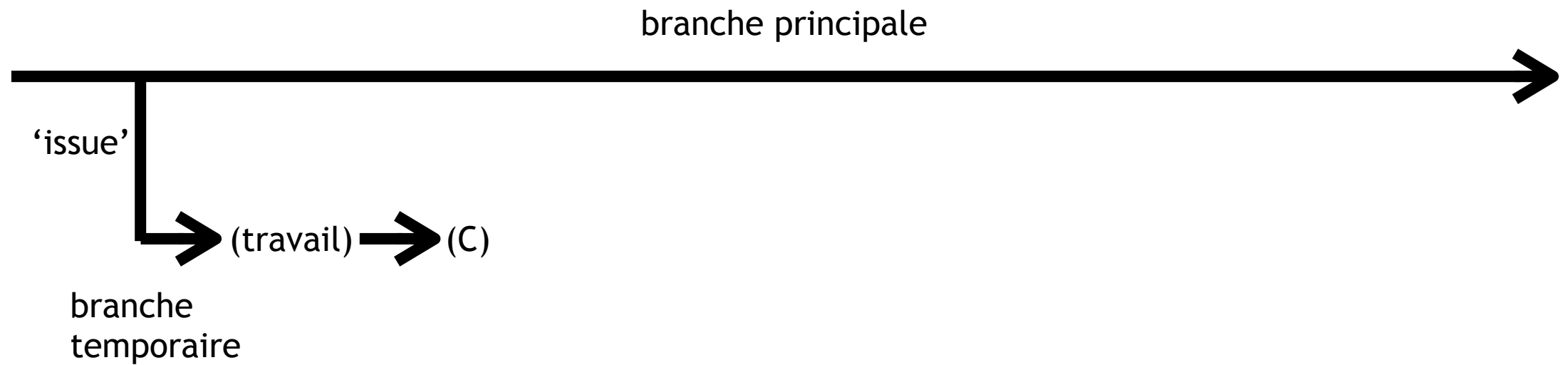
Stratégie de branchement de GitHub



Stratégie de branchement de GitHub

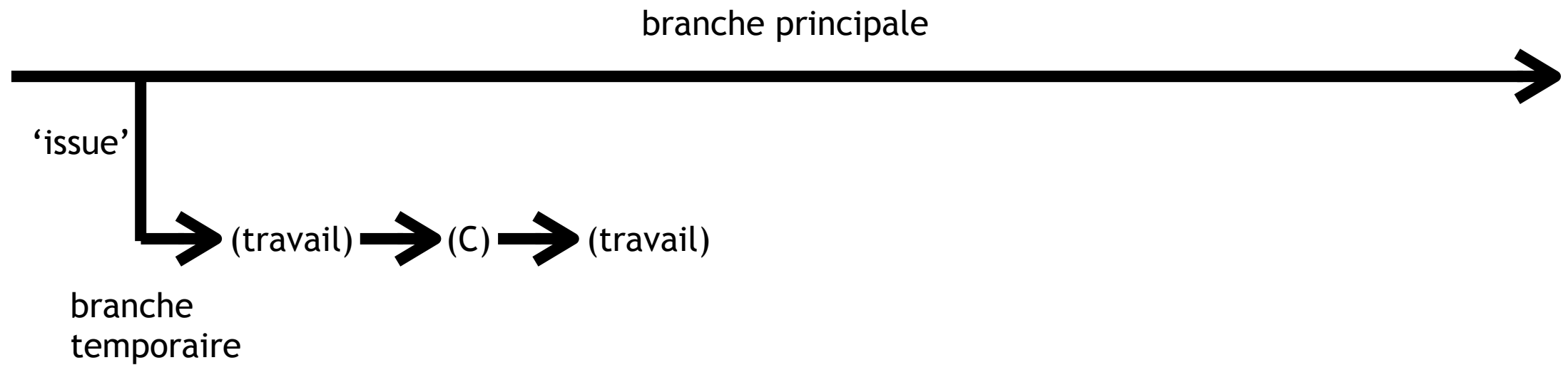


Stratégie de branchement de GitHub



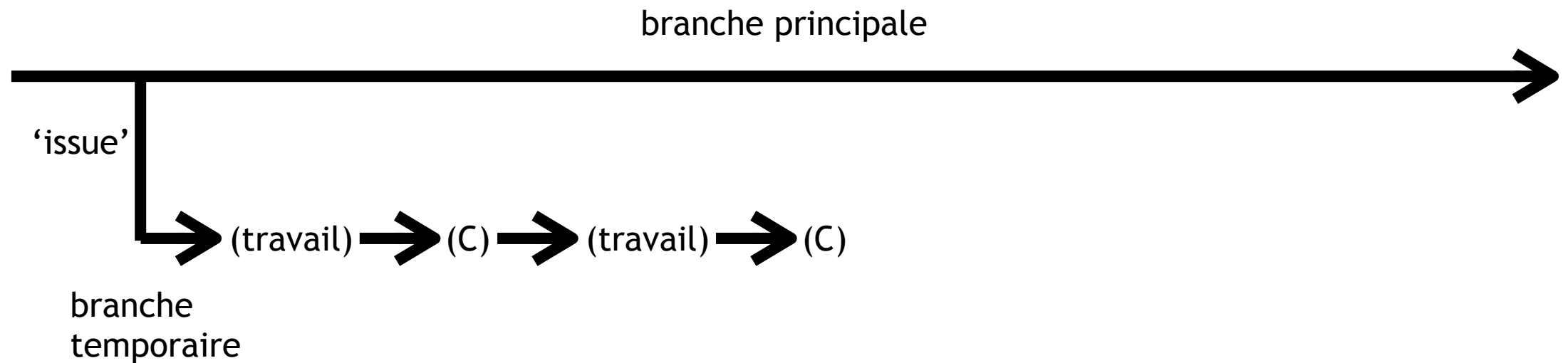
C : Commit

Stratégie de branchement de GitHub



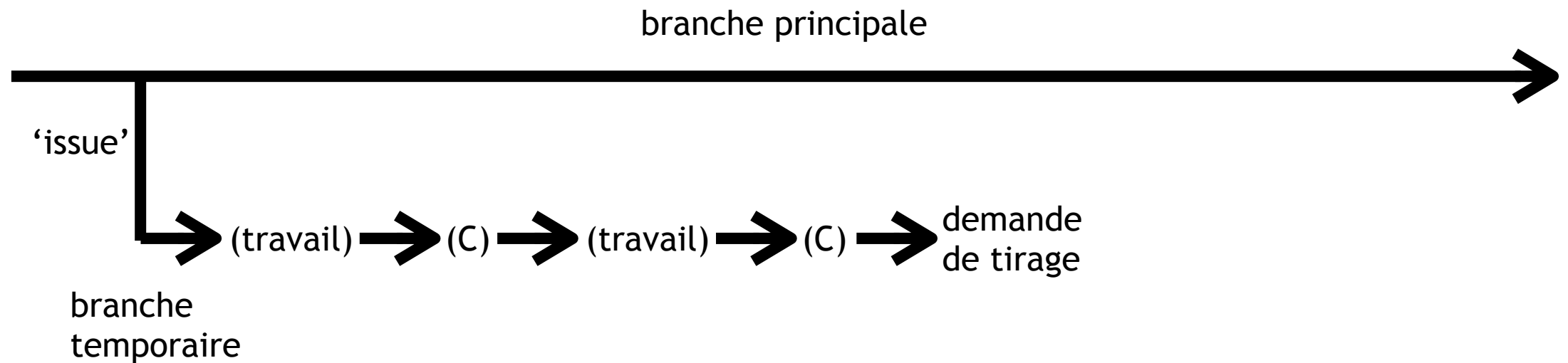
C : Commit

Stratégie de branchement de GitHub



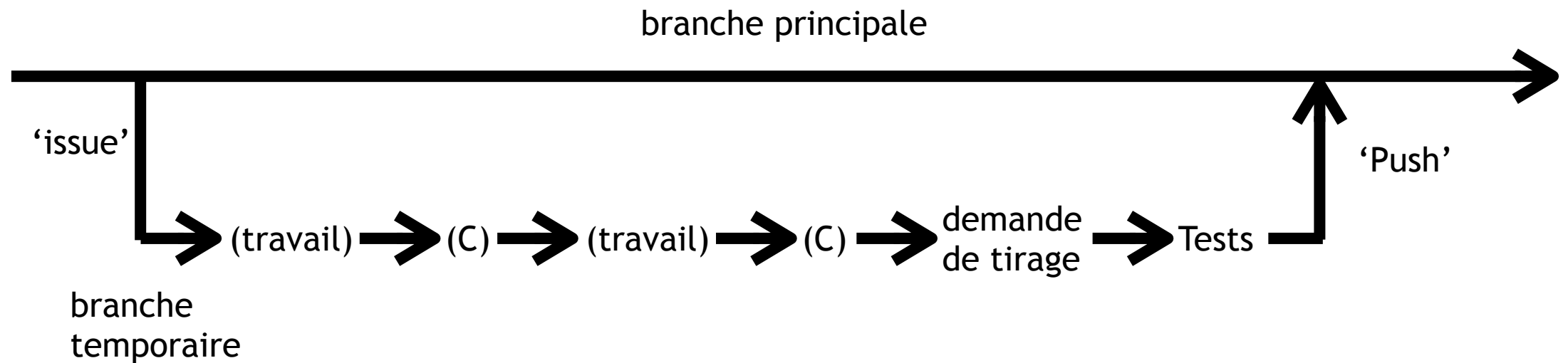
C : Commit

Stratégie de branchement de GitHub



C : Commit

Stratégie de branchement de GitHub



C : Commit

Stratégie de branchement de GitLab



Stratégie de branchement de GitLab



- Basé sur plusieurs branches permanentes ayant des fonctionnalités différentes, et des branches temporaires
 - branche « production » : branche finale qui produit un logiciel déployable et exploitable ("stable" et "complet").
 - Il est interdit de travailler directement dessus.
 - Elle est permanente.

Stratégie de branchement de GitLab



- Basé sur plusieurs branches permanentes ayant des fonctionnalités différentes, et des branches temporaires
 - Une ou plusieurs branches « pré-production » : existents pour faire des tests dans des environnements spécifiques.
 - Il est interdit de travailler directement dessus.
 - Elles sont permanentes.

Stratégie de branchement de GitLab



- Basé sur plusieurs branches permanentes ayant des fonctionnalités différentes, et des branches temporaires
 - La branche 'main' (ou 'master') est la branche de développement du code.
 - Il est interdit de travailler directement dessus.
 - Elle est permanente.

Stratégie de branchement de GitLab



- Basé sur plusieurs branches permanentes ayant des fonctionnalités différentes, et des branches temporaires
 - chaque branche temporaire correspond à une demande de nouvelle fonctionnalité, de correction ou d'amélioration.
 - Elles peuvent être créées sur n'importe quelle branche permanente ou temporaire

Stratégie de branchement de GitLab



branche production



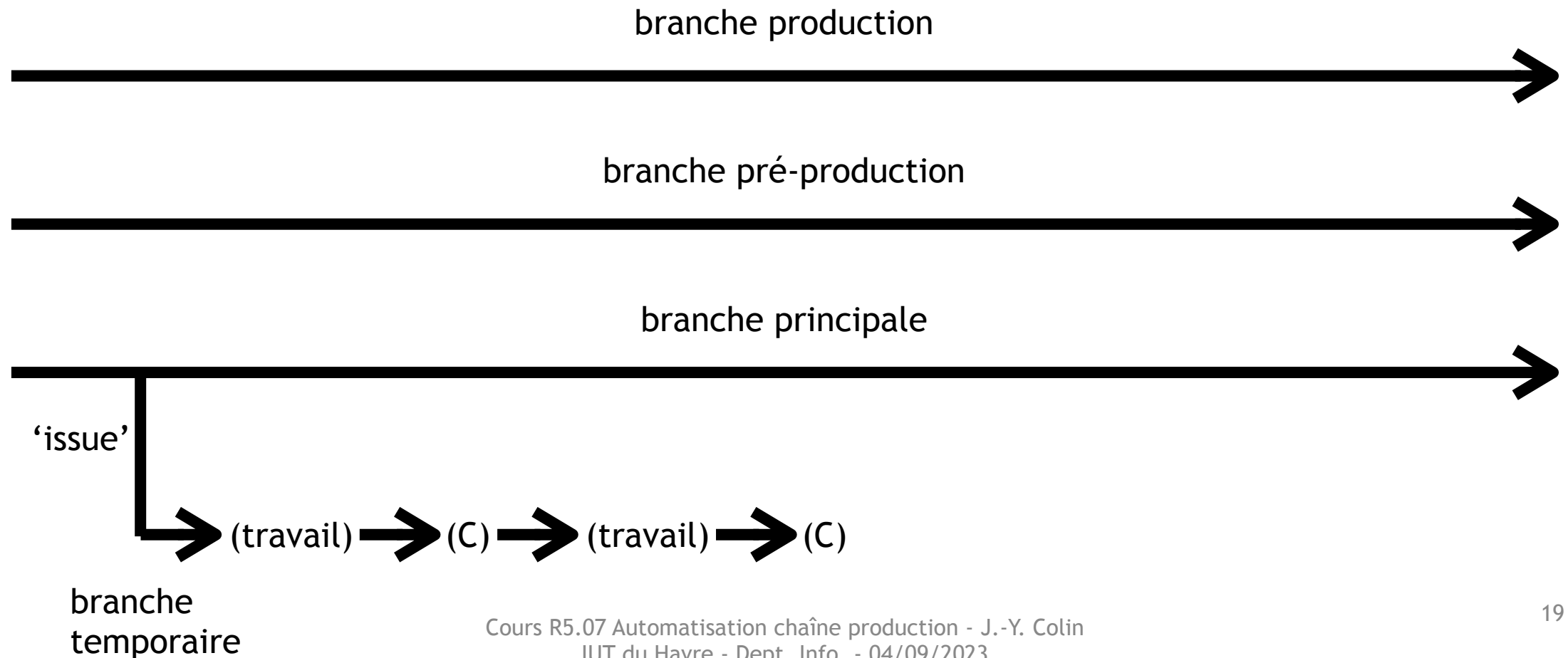
branche pré-production



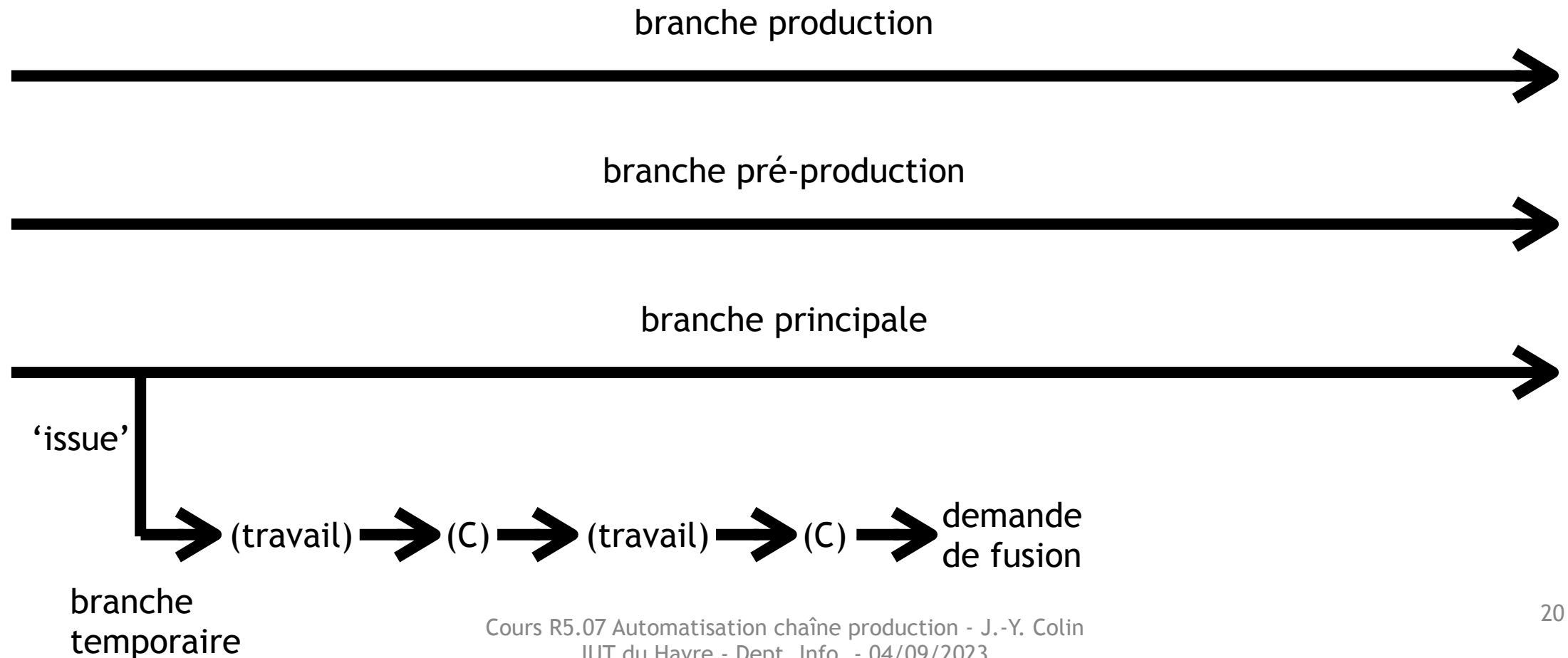
branche principale



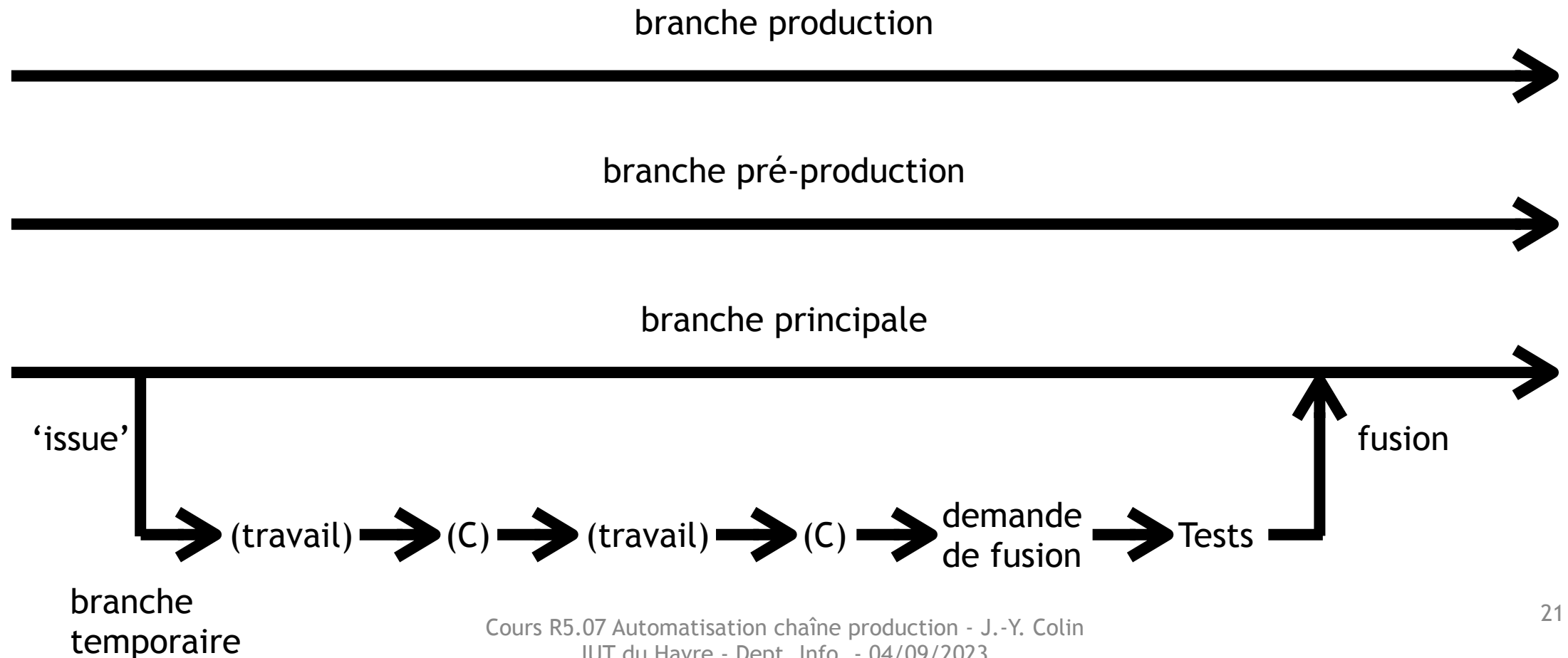
Stratégie de branchement de GitLab



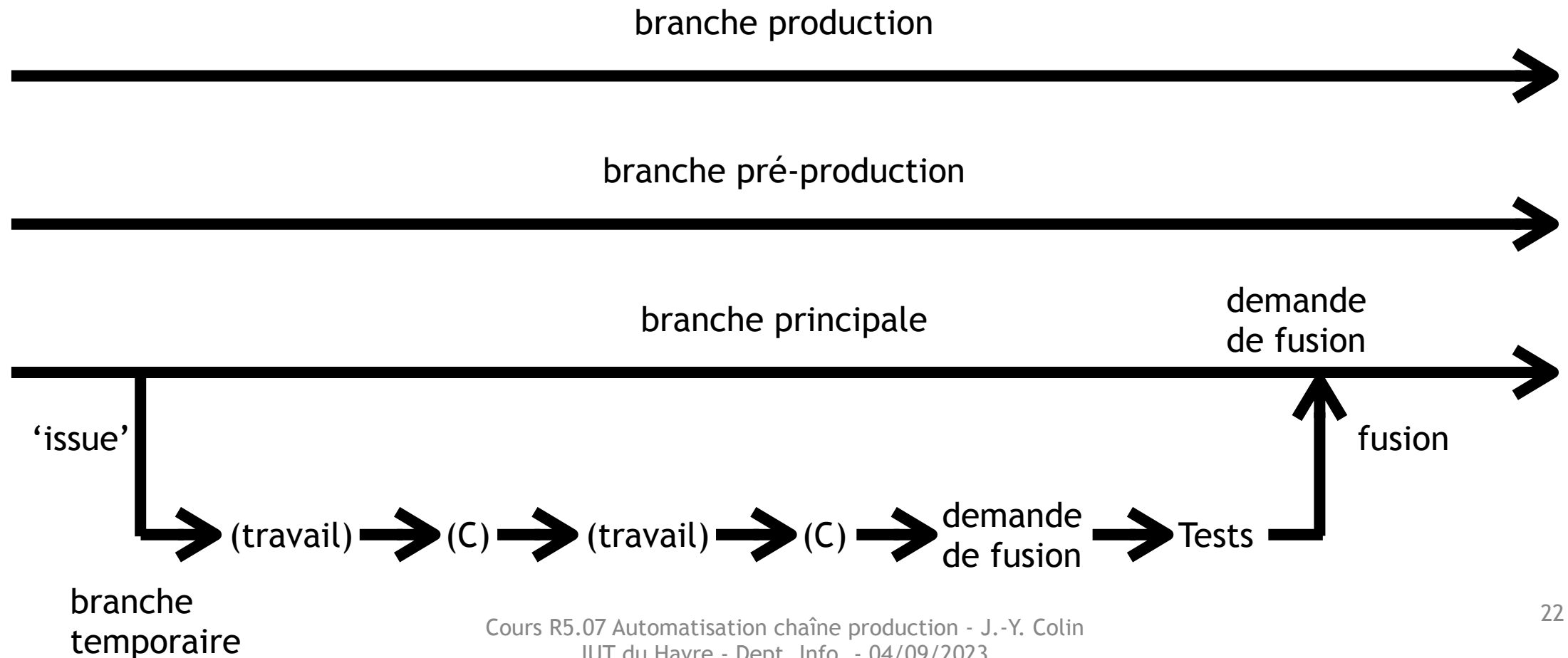
Stratégie de branchement de GitLab



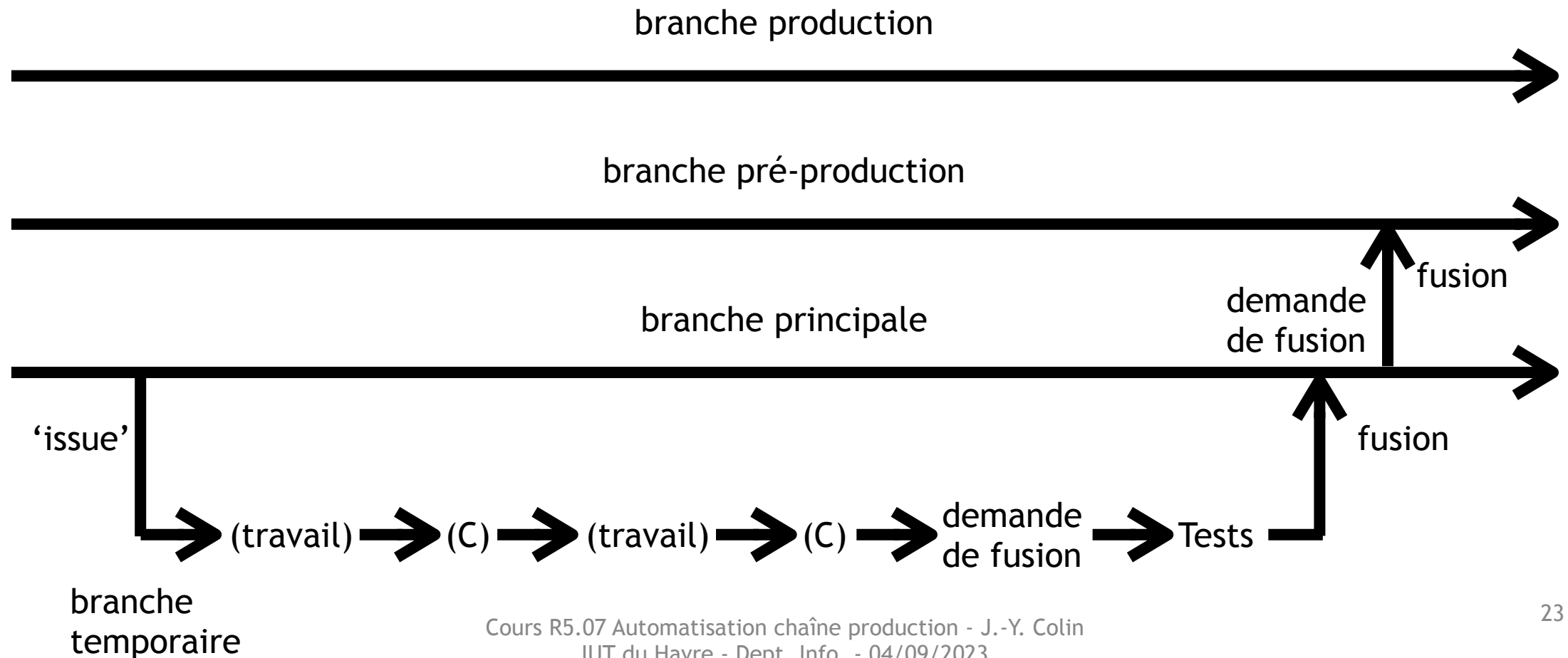
Stratégie de branchement de GitLab



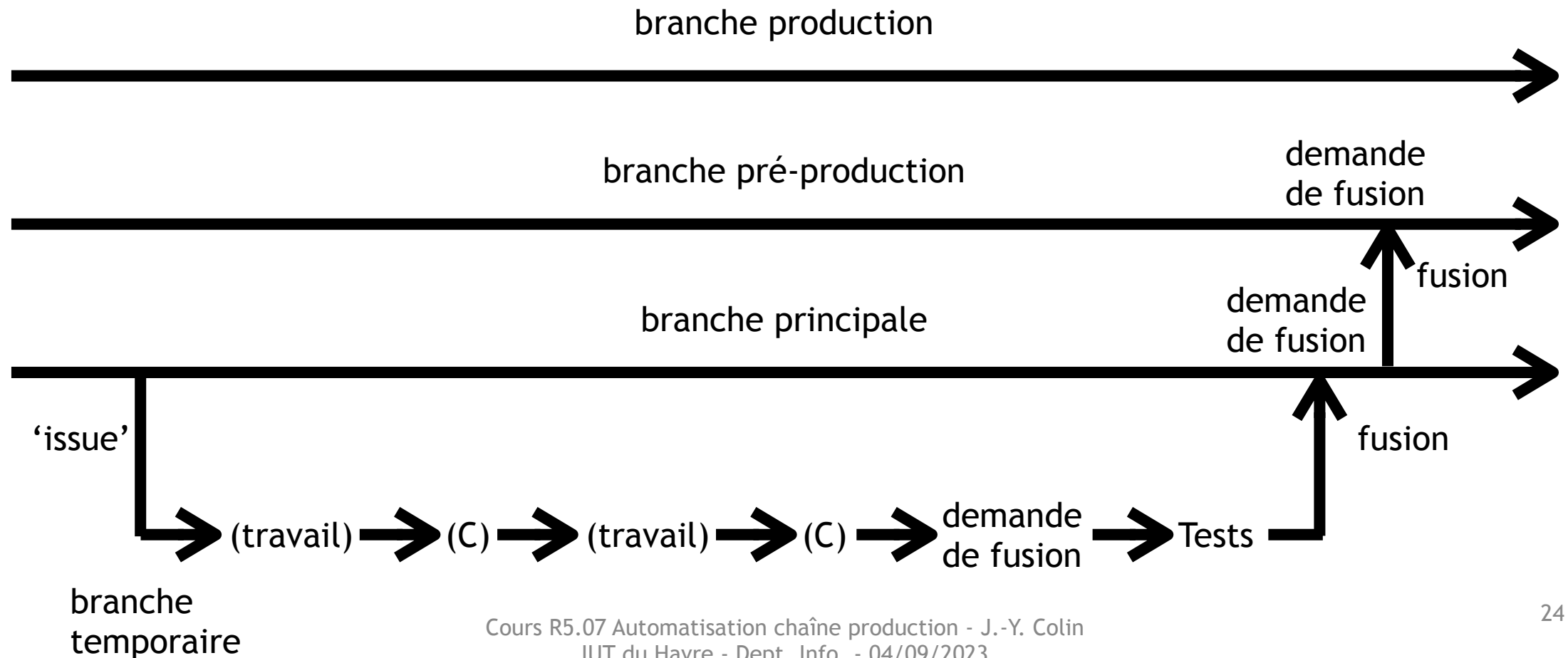
Stratégie de branchement de GitLab



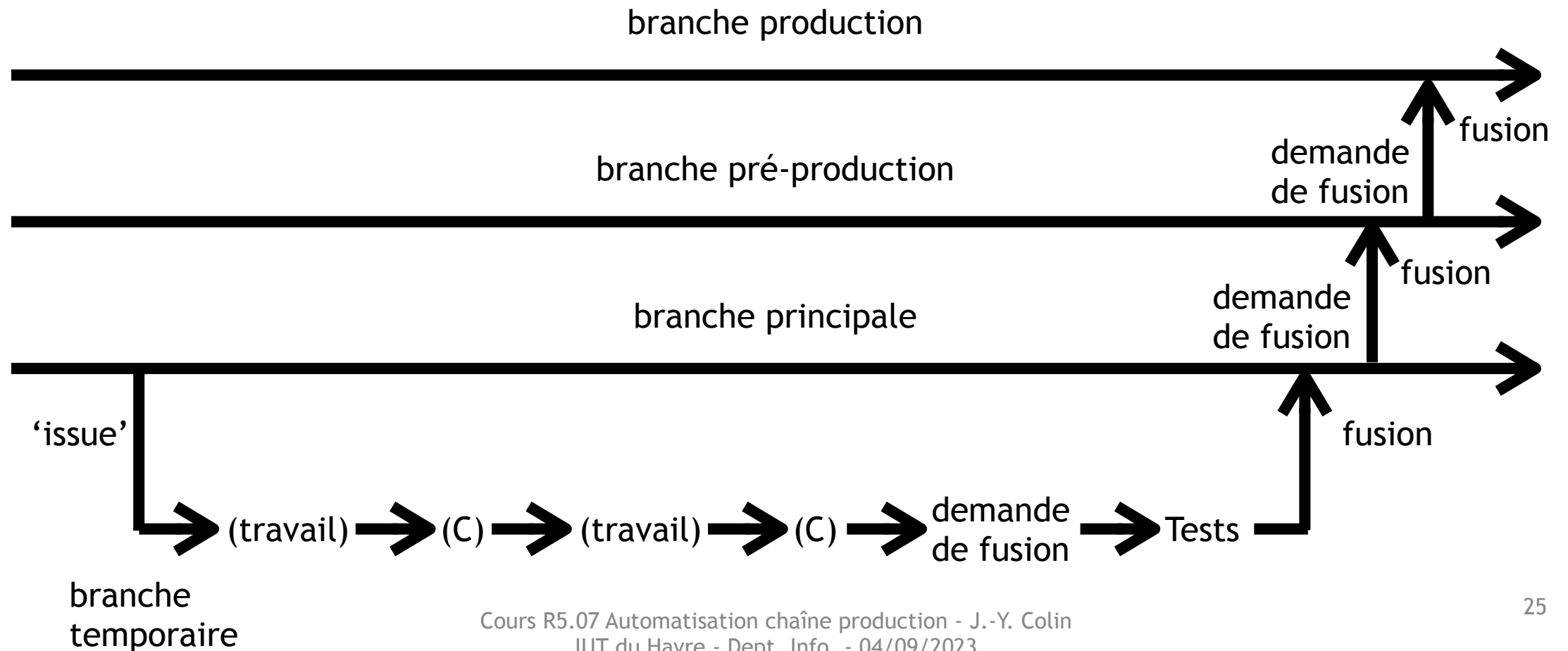
Stratégie de branchement de GitLab



Stratégie de branchement de GitLab



Stratégie de branchement de GitLab



Stratégie de branchement de GitLab



- Elle applique une politique 'upstream-first' :
 - impose que tous les merges se font dans le sens 'feature' -> 'main' -> "pré-production" (tous) -> "production" (cf. flèches toujours vers le haut)
 - en cas de bug dans la phase de test dans une branche "pré-production", on repart d'une nouvelle branche 'feature' à partir de la branche 'main'.
 - Toute modification re-suivra le chemin -> préproduction -> production

Stratégie de branchement de GitLab



branche production



branche pré-production

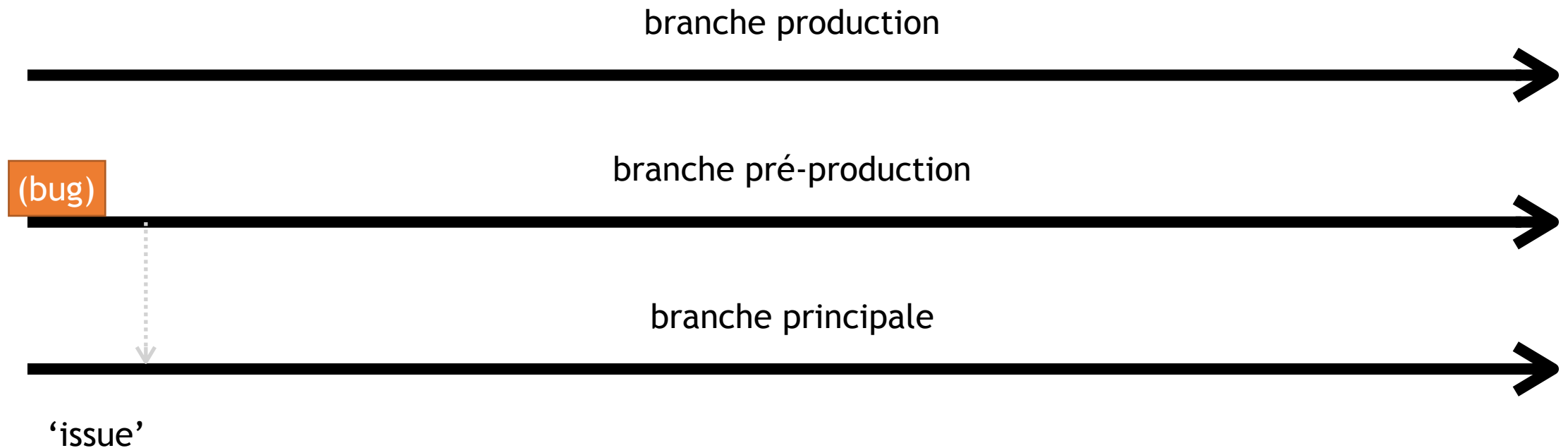
(bug)



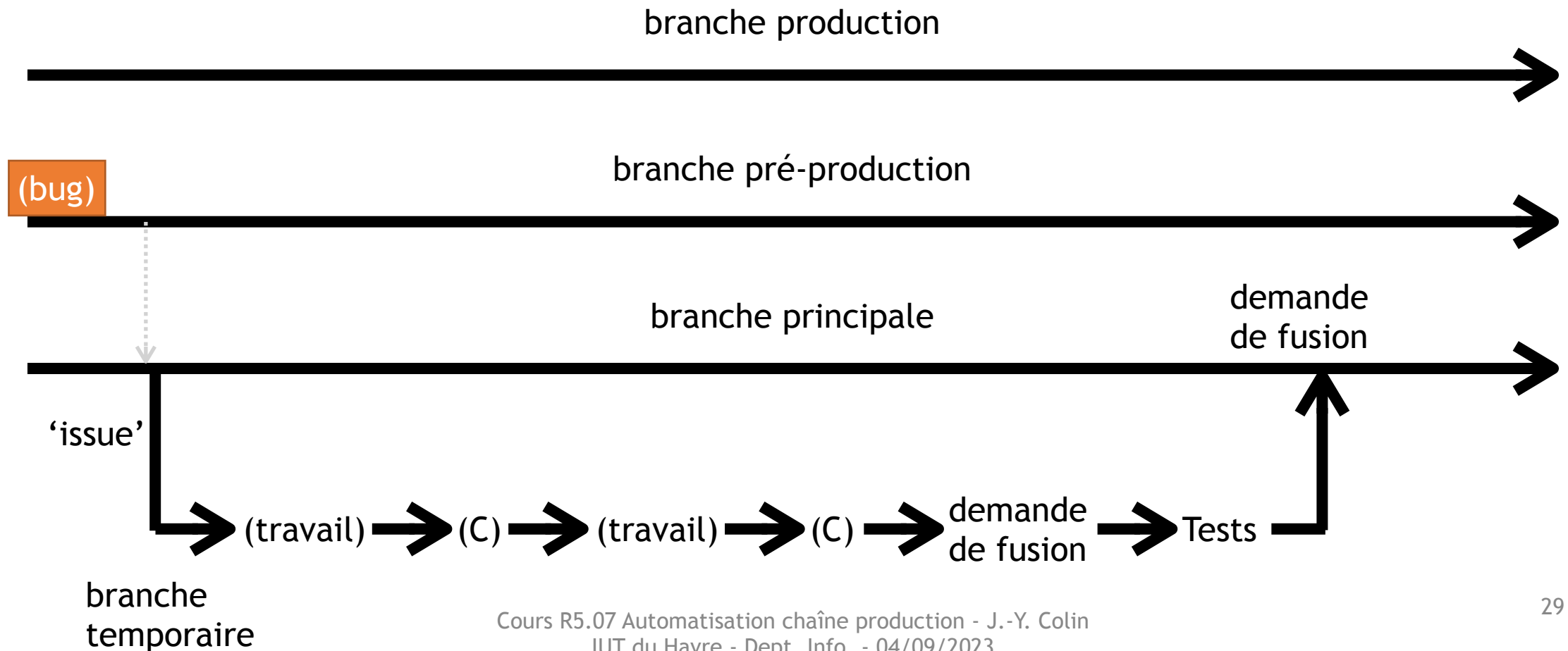
branche principale



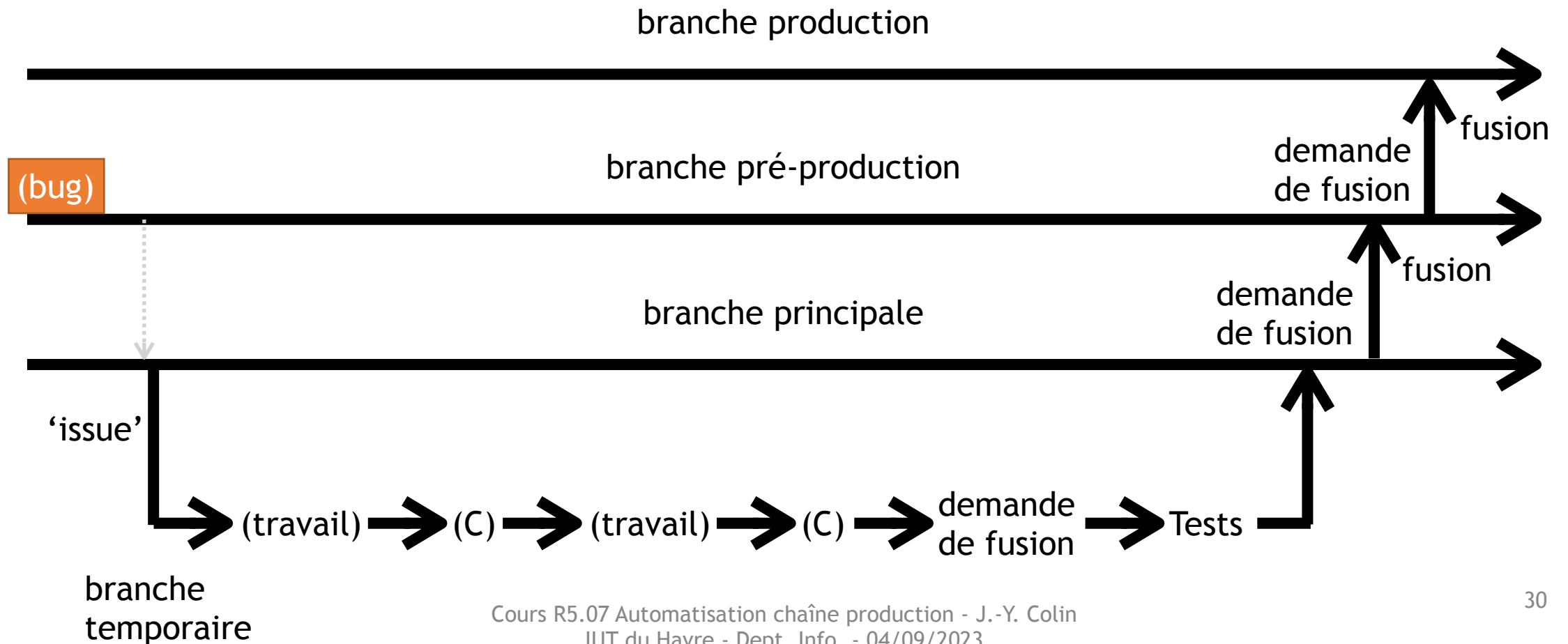
Stratégie de branchement de GitLab



Stratégie de branchement de GitLab



Stratégie de branchement de GitLab

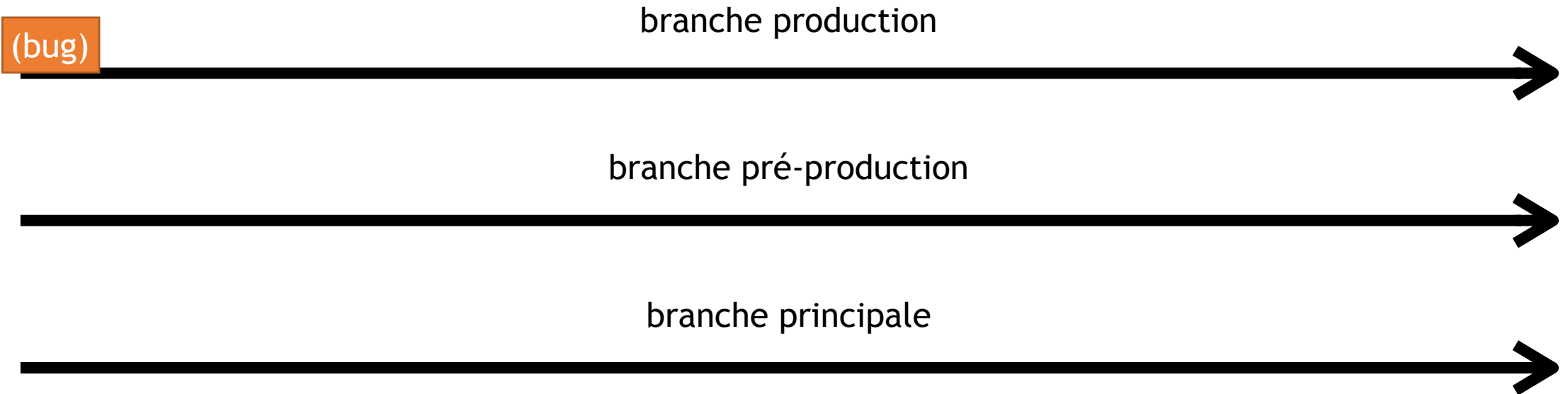


Stratégie de branchement de GitLab

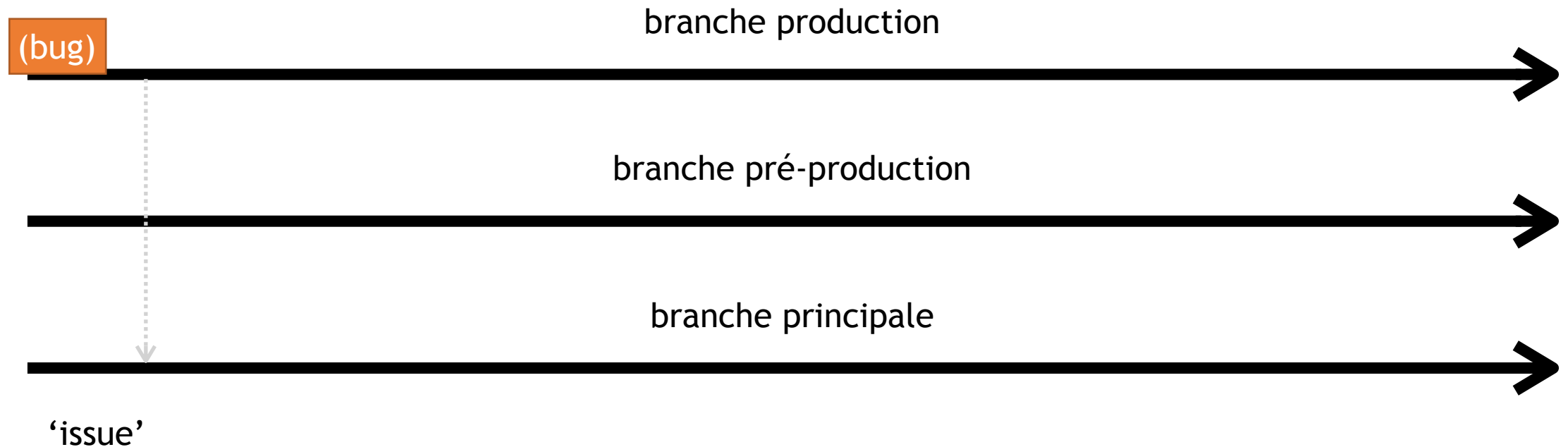


- Elle applique une politique 'upstream-first' (fin) :
 - idem pour un bug dans la branche "production" (déployée et exploitée) : le bug sera traité dans une nouvelle branche 'feature' de la branche 'main'.
 - Note : étant donné qu'un bug découvert seulement dans la branche production est généralement très urgent ('hotfix'), il est fortement conseillé de demander un 'Merge Request' immédiat vers les branches "pré-production" et "production", dès que la branche est fusionnée dans la branche 'main' (pour ne pas avoir à attendre une future fusion de la branche 'main' vers les autres branches, qui pourrait n'être planifiée que bien plus tard).

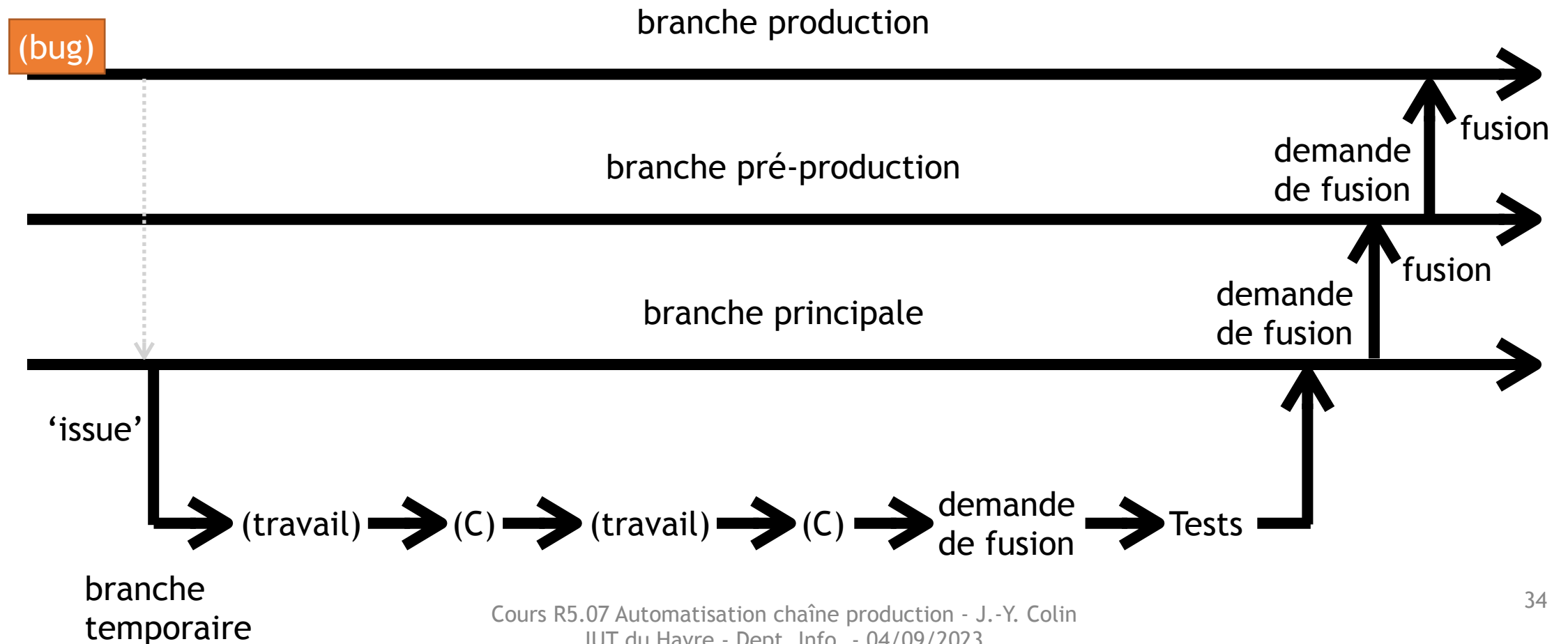
Stratégie de branchement de GitLab



Stratégie de branchement de GitLab



Stratégie de branchement de GitLab

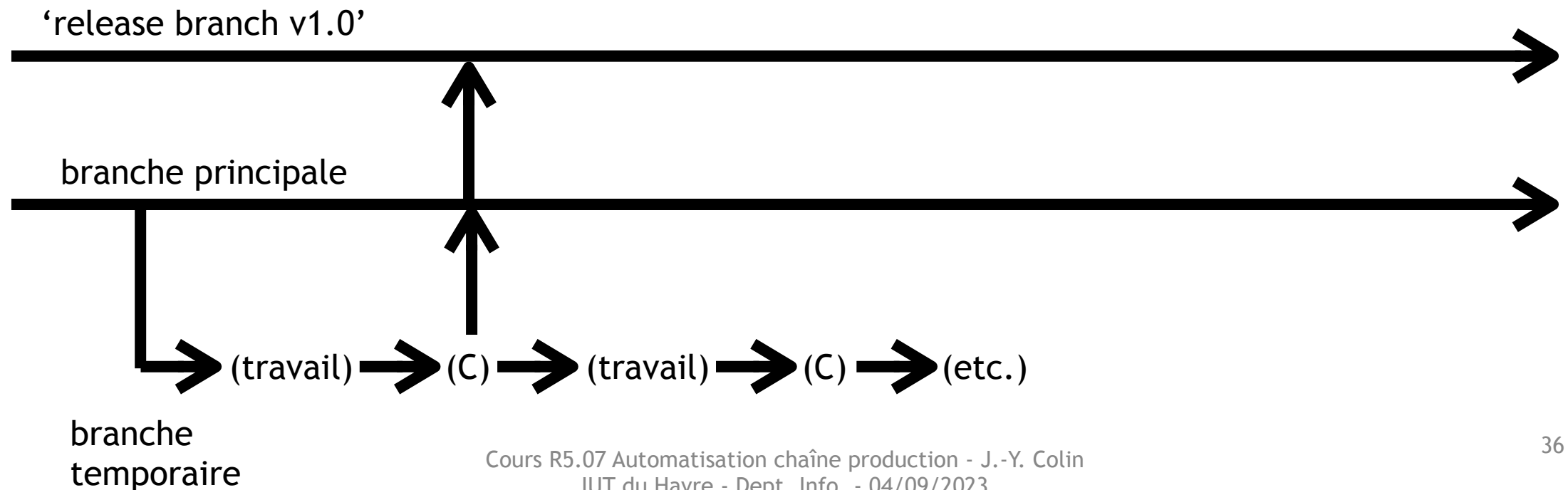


Stratégie de branchement de GitLab

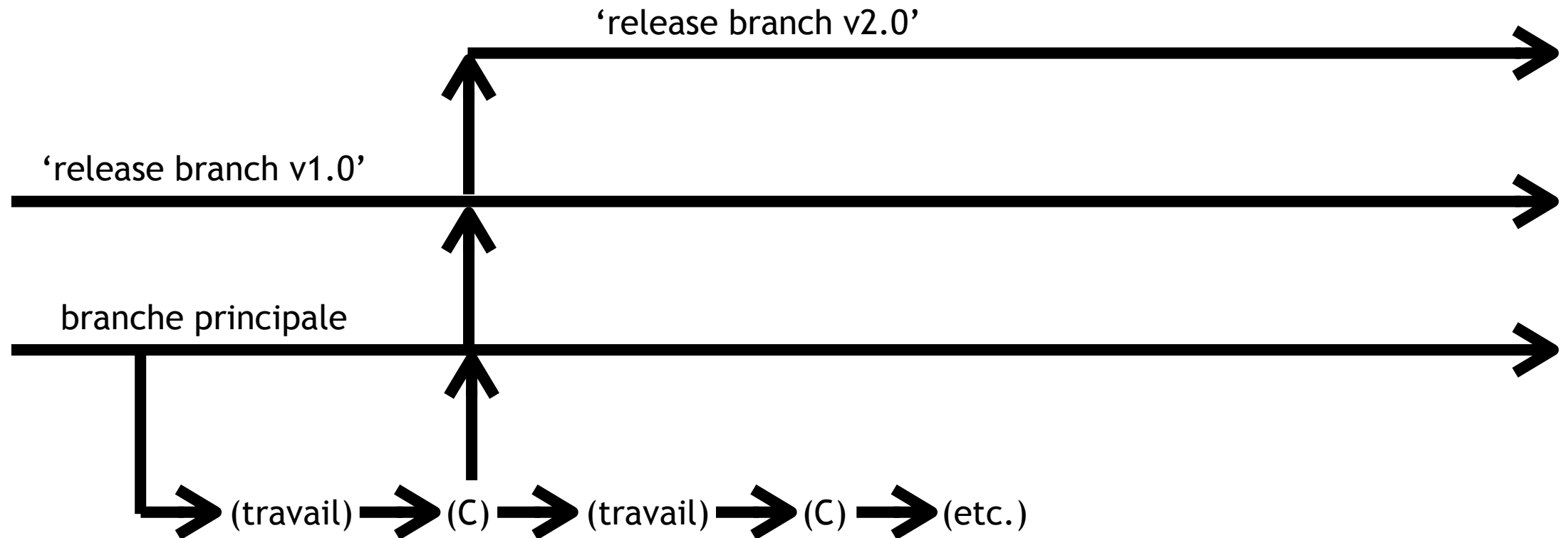


- Variante très courante : Au lieu d'avoir une branche permanente Production, il y a à la place plusieurs branches long-terme correspondant à des 'Releases' (« versions »)

Stratégie de branchement de GitLab

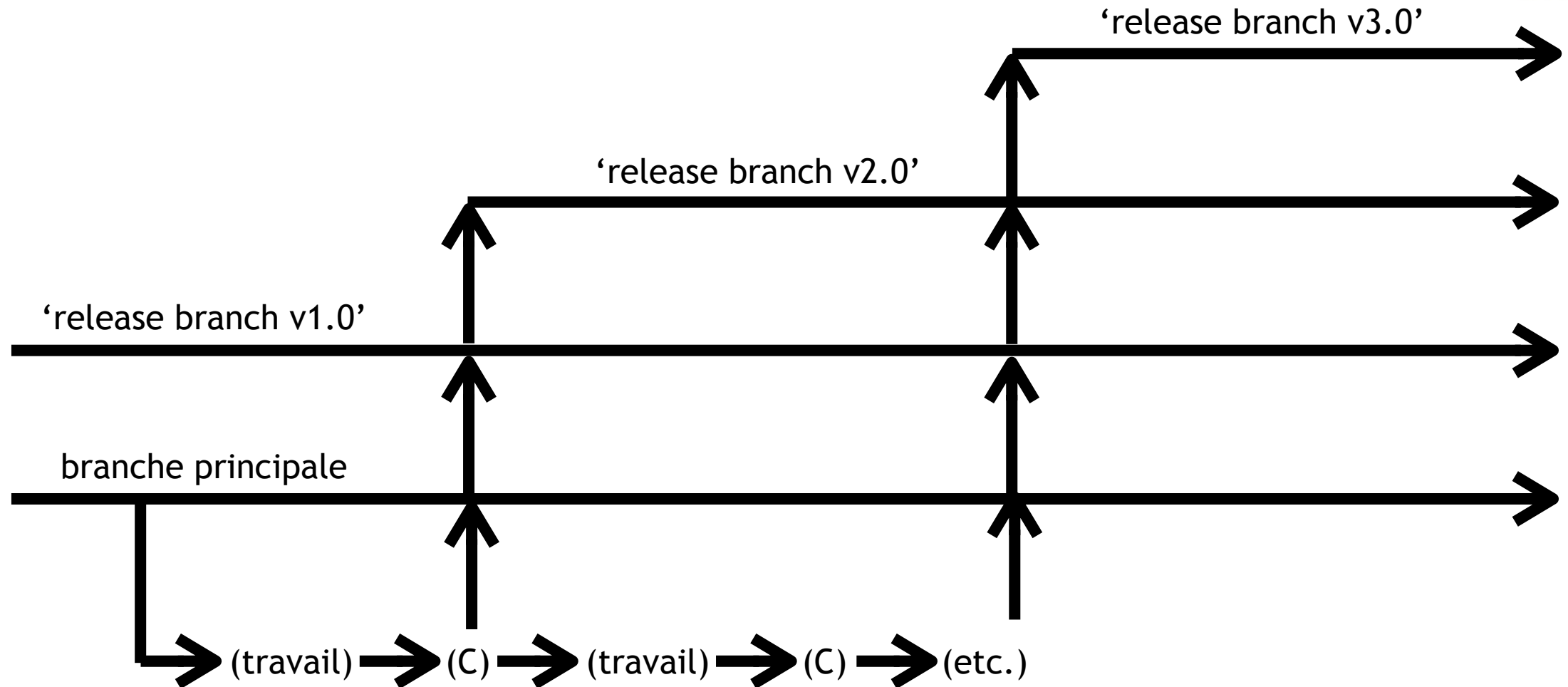


Stratégie de branchement de GitLab



branche
temporaire

Stratégie de branchement de GitLab



branche
temporaire

Stratégie de branchement de GitLab



- Les branches de versions sont souvent qualifiées de :
 - obsolètes : les modifications dans la branche principale ne seront généralement plus appliquées à cette branche
 - ‘Long Term’ (‘LTR’) : les modifications dans la branche principale seront appliquées pour une durée de quelques années, et cette version sera considérée comme « très fiable »
 - ‘edge’ / ‘latest’ : version la plus récente pour les utilisateurs, considérée comme « fiable » (edge) ou « très fiable » (latest)
 - beta : pour les testeurs, « peu fiable »
 - ‘nightly build’ / alpha : pour les testeurs, « incomplète »

Stratégie de branchement de GitLab

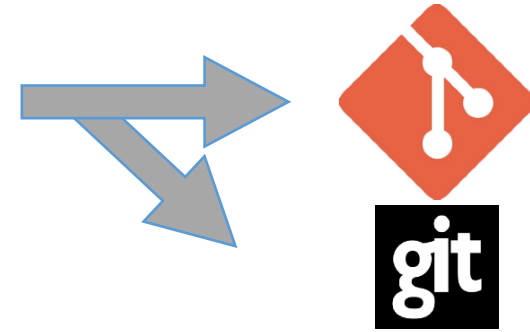


- le classement des branches en « versions » est généralement utilisée pour les logiciels publiés pour le grand public

Stratégie de branchement de Git

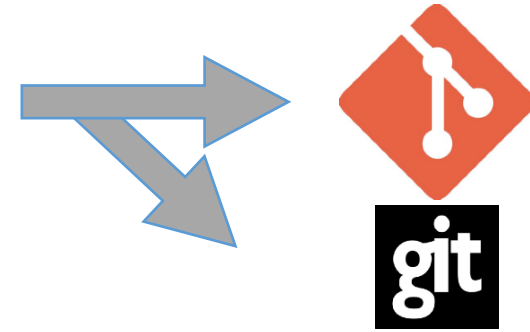


Stratégie de branchement de Git



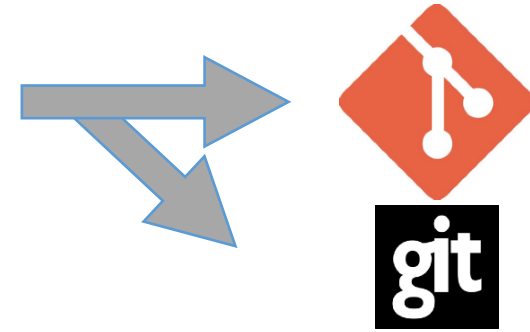
- Git est le plus complexe : basé sur une branche principale ('main') (visible des utilisateurs) qui est la version déployée et exploitée,
 - plus une branche permanente de développement ('develop') (visible uniquement en interne, en principe),)
 - plus des branches ponctuelles ('feature branches') de la branche de développement qui portent le travail de développement logiciel

Stratégie de branchement de Git



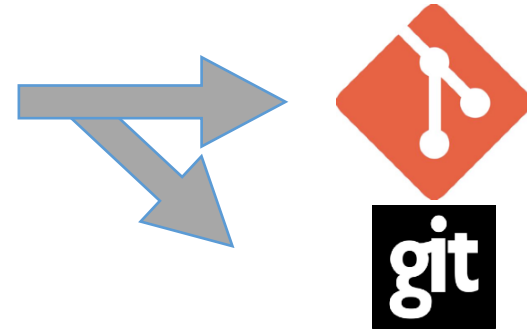
- plus des branches ponctuelles ('release branches') de la branche de développement qui ont pour but de porter le travail de test et de débogage final (avant déploiement)
- plus des branches ponctuelles ('hotfix branches') de la branche principale, au cas où des bugs sont apparus en exploitation de la branche principale

Stratégie de branchement de Git

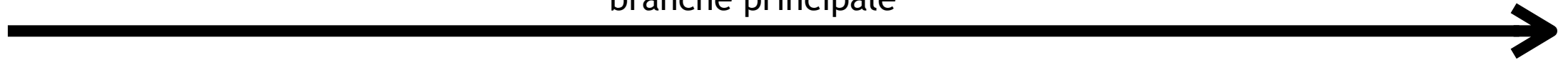


- le Git flow commence avec deux branches permanentes :
 - la branche permanente 'main', qui conserve un code validé ('stable')
 - plus la branche permanente 'develop', sur laquelle le travail de développement normal est fait avec des sous-branches 'feature'
 - ...

Stratégie de branchement de Git



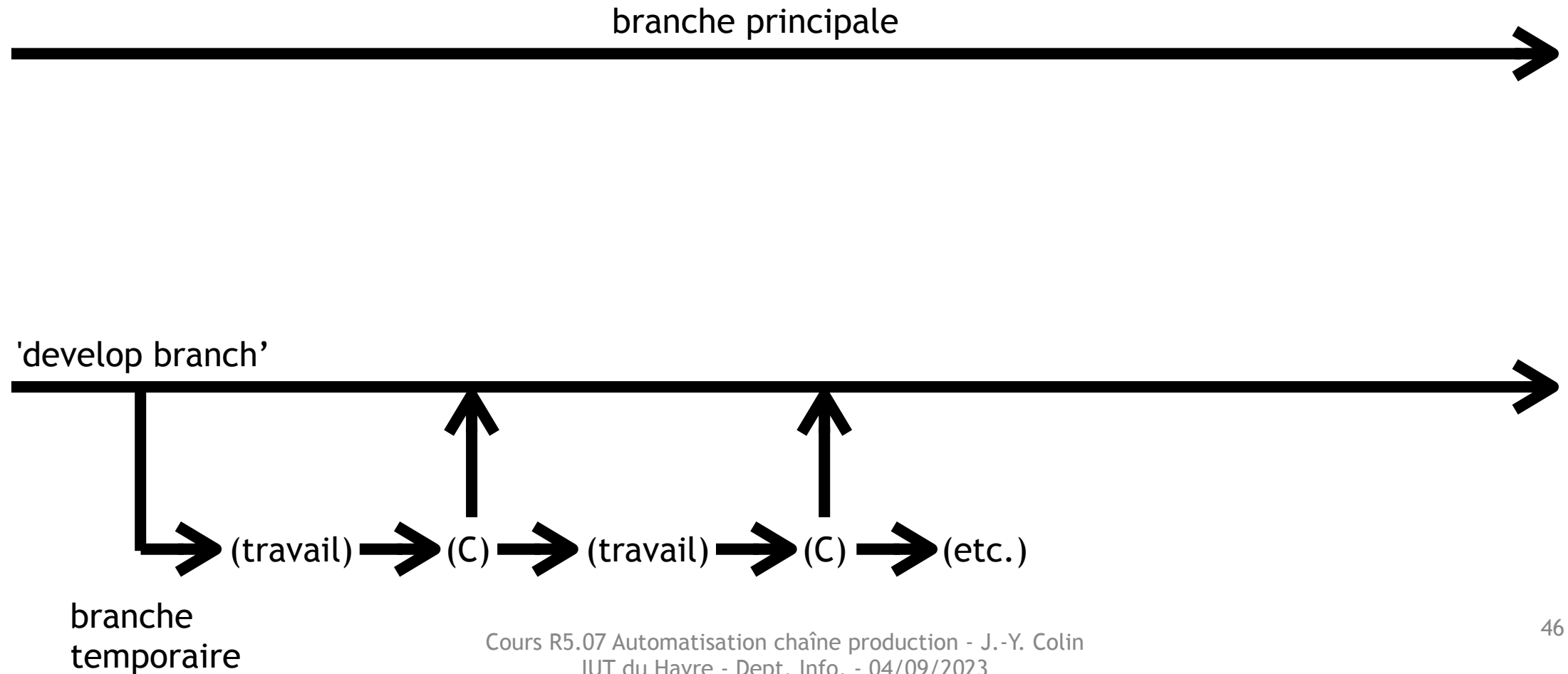
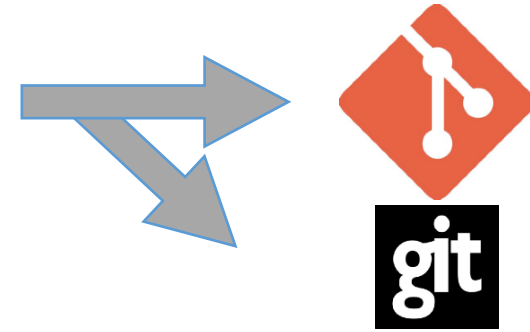
branche principale



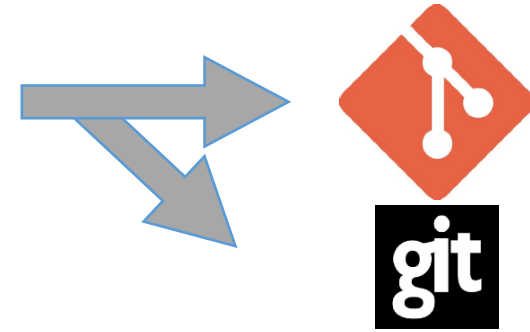
'develop branch'



Stratégie de branchement de Git

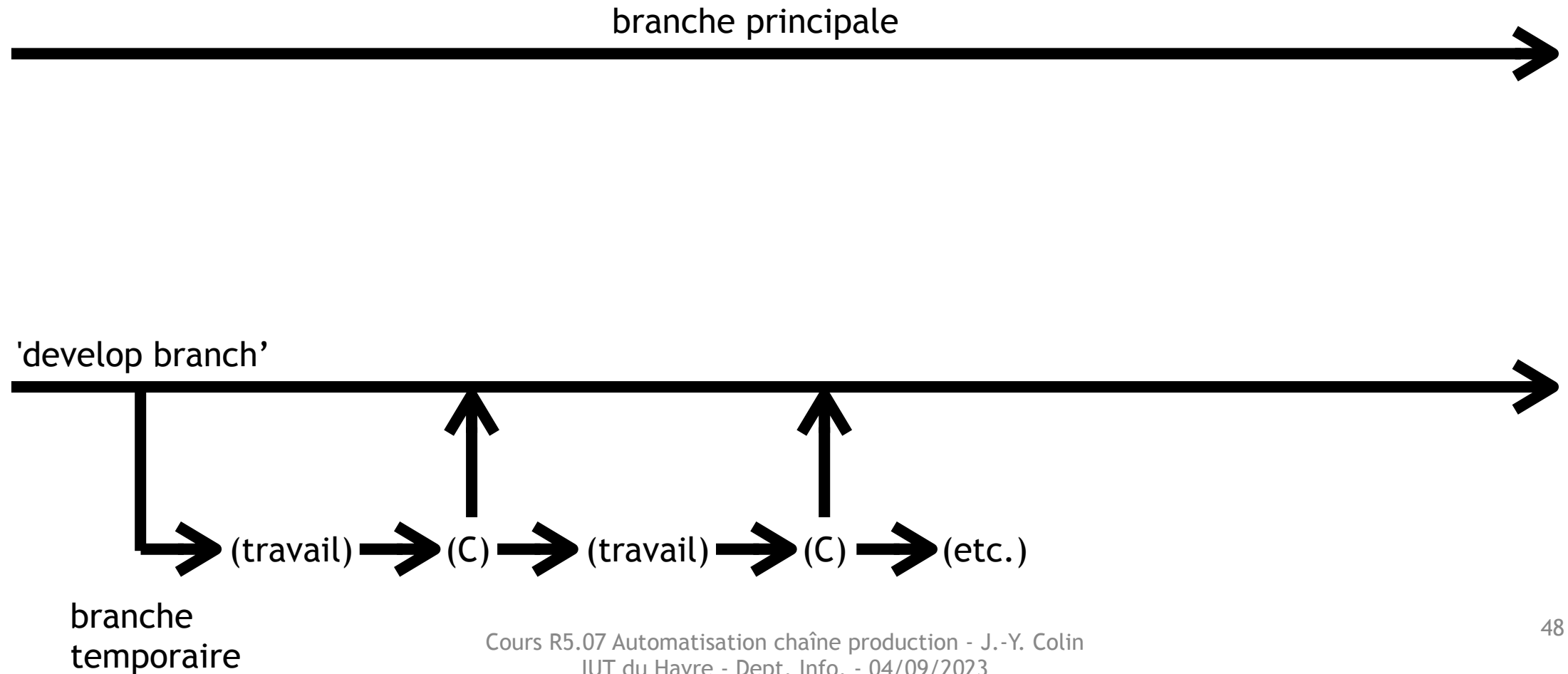
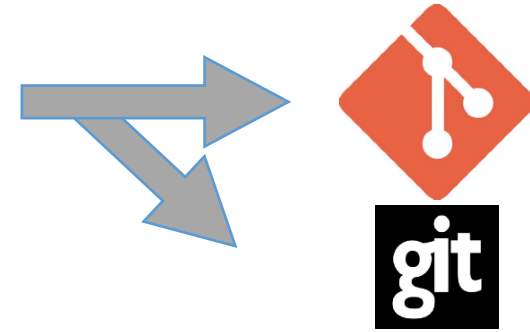


Stratégie de branchement de Git

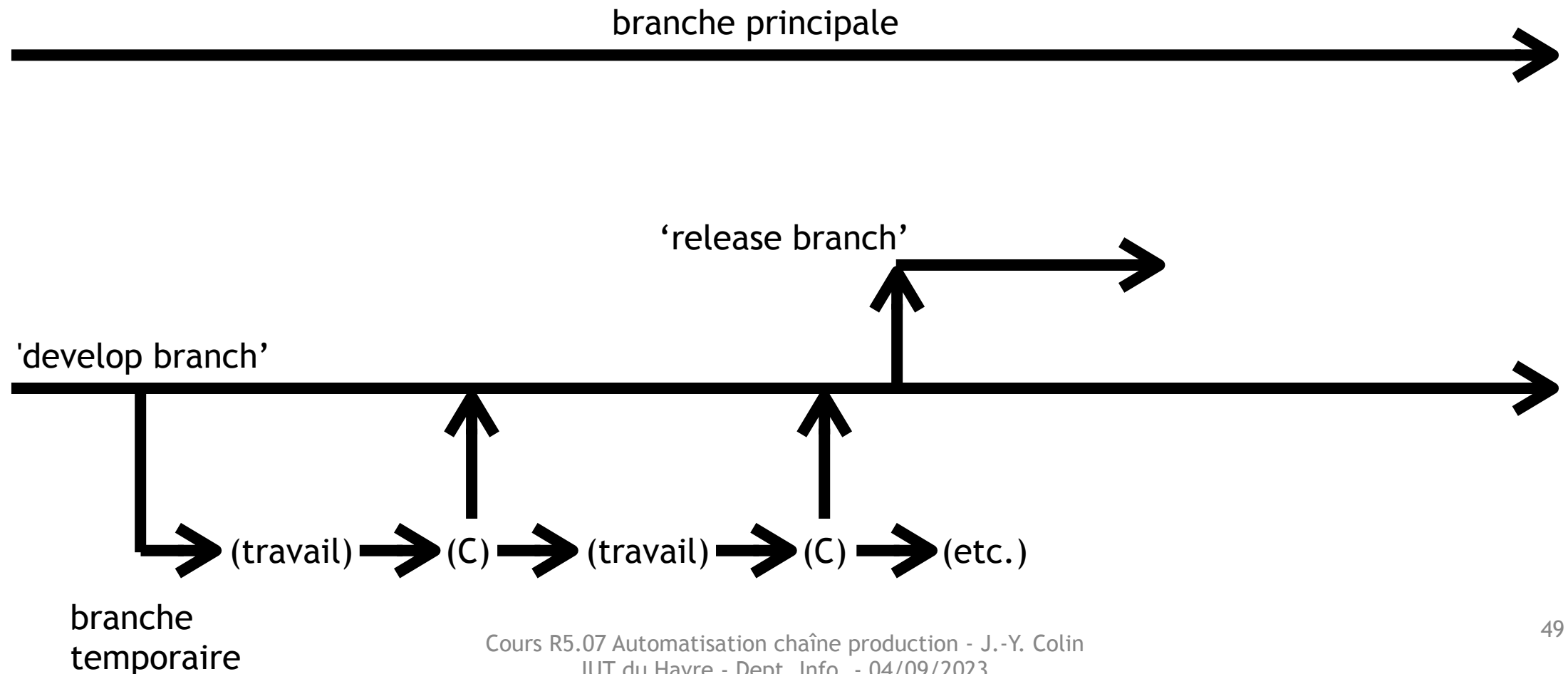
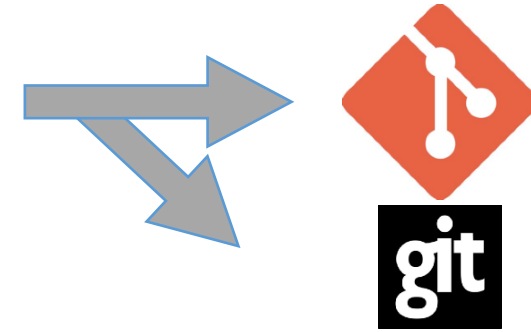


- le Git flow continue avec une branche ‘release’ à partir de la branche ‘develop’,
 - puis parfois avec des branches ‘release candidates’

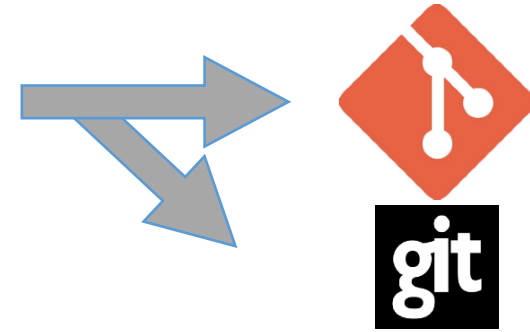
Stratégie de branchement de Git



Stratégie de branchement de Git

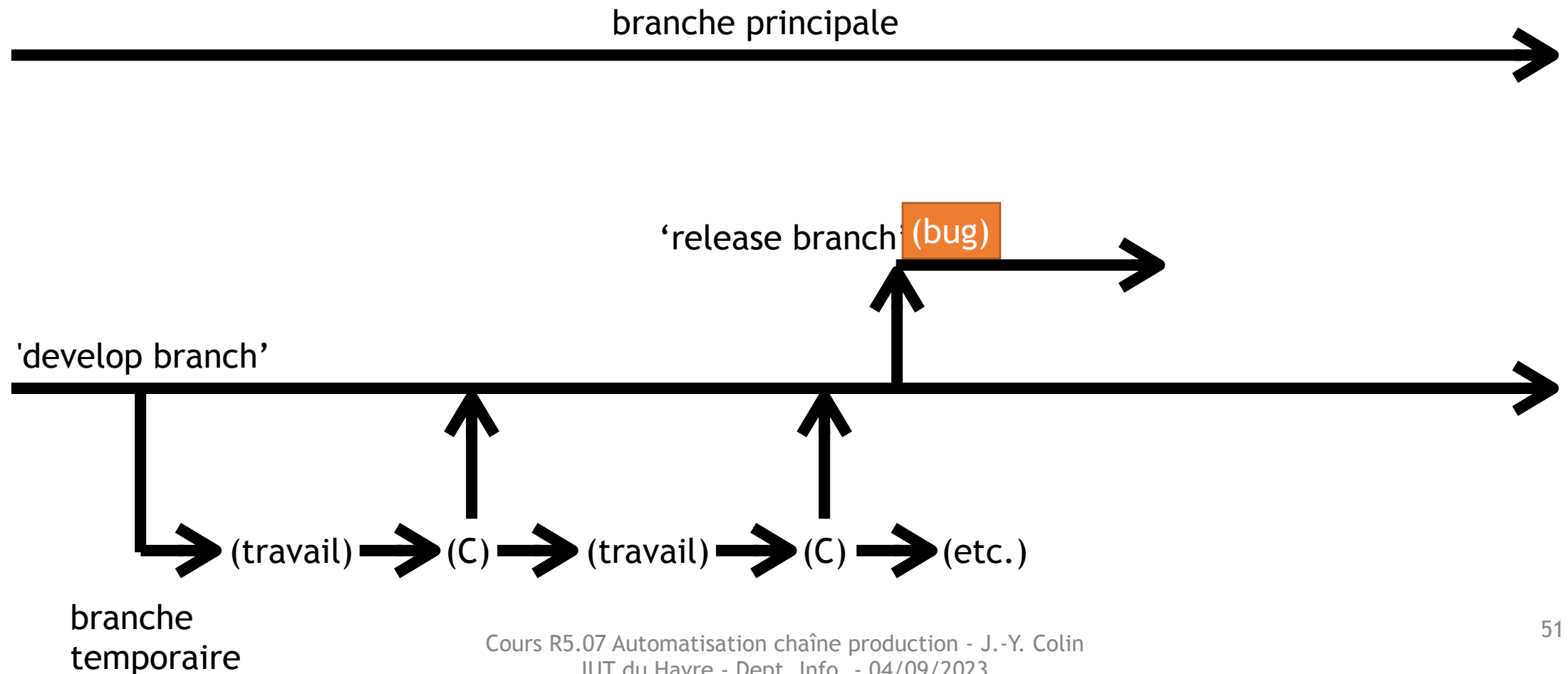
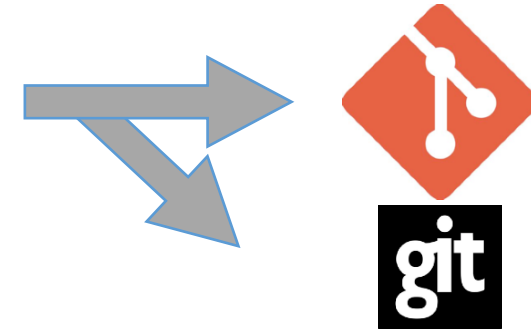


Stratégie de branchement de Git

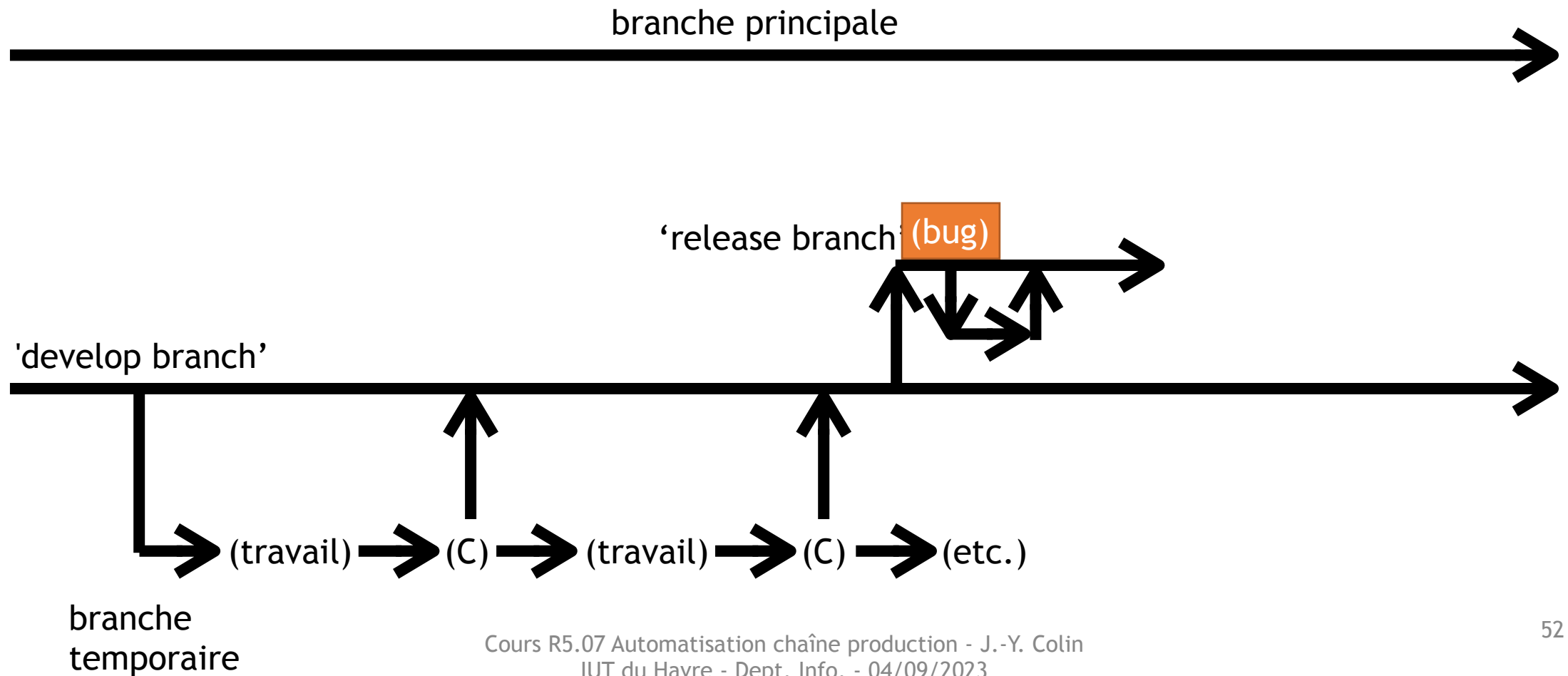
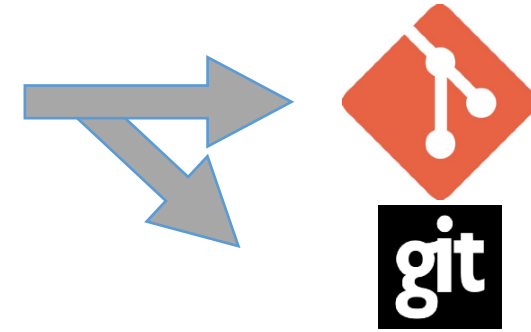


- en cas de bug dans ces branches, les corrections sont faites dans des sous-branches temporaires de la branche 'release' (pas de la branche 'develop')
- puis les corrections dans la branch 'release' sont fusionnées dans la branch 'develop' pour la mettre à jour

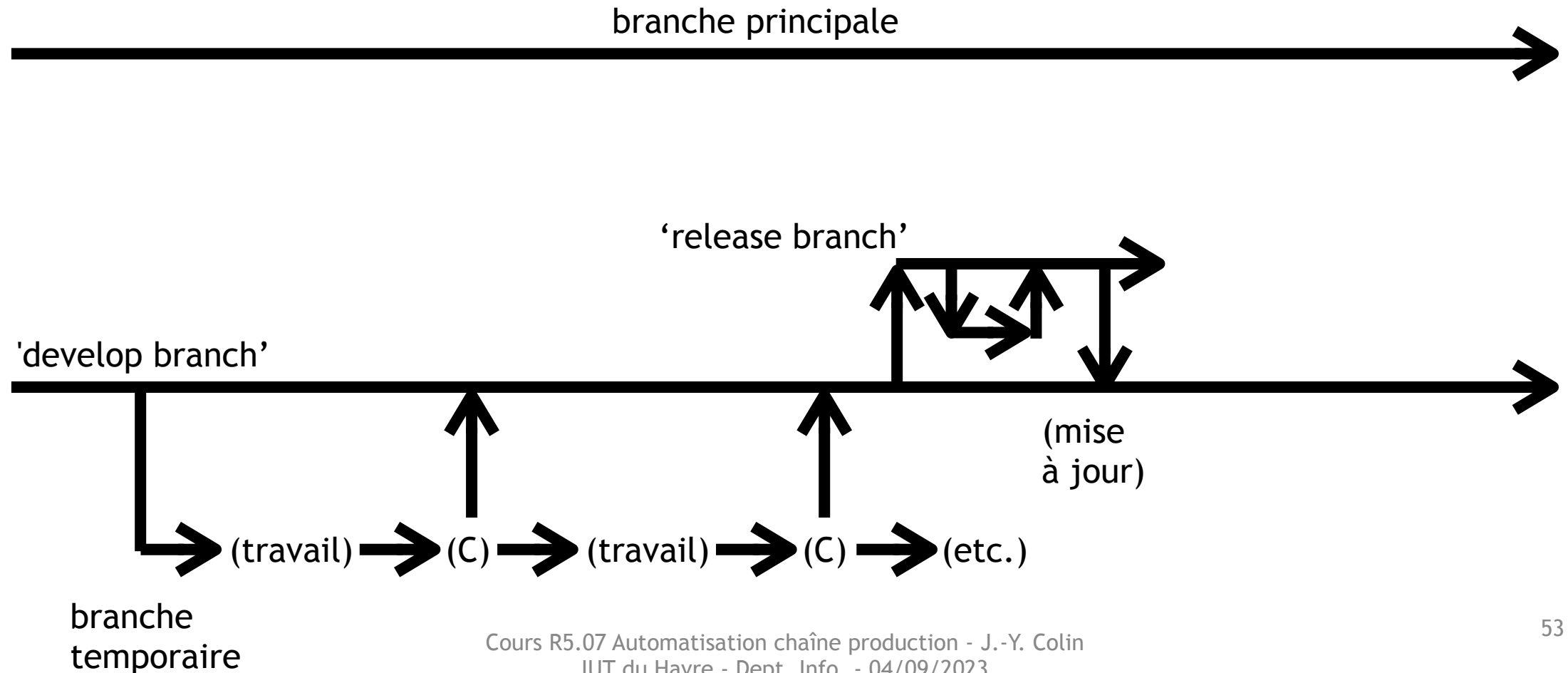
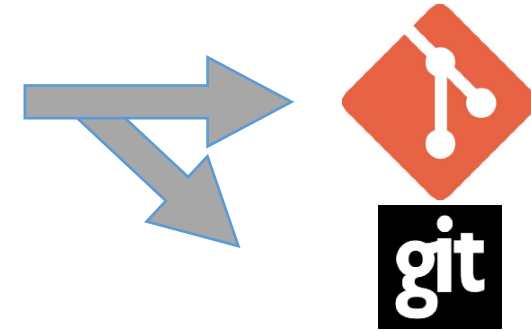
Stratégie de branchement de Git



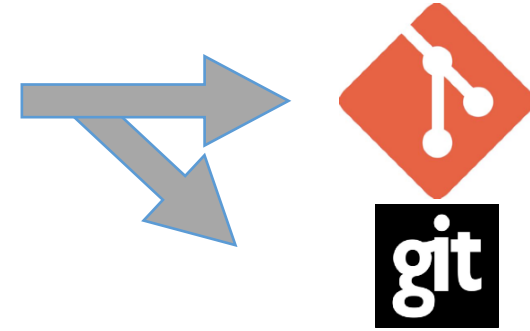
Stratégie de branchement de Git



Stratégie de branchement de Git

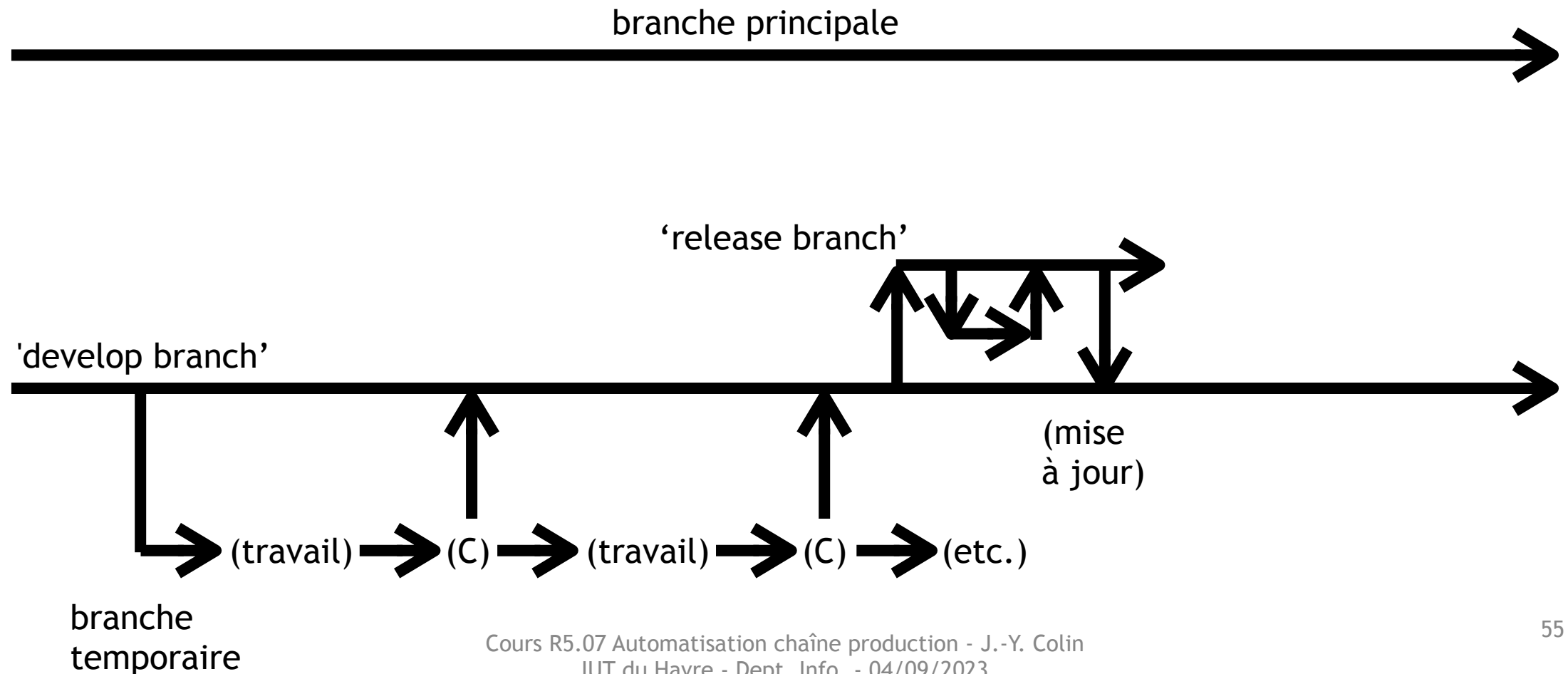
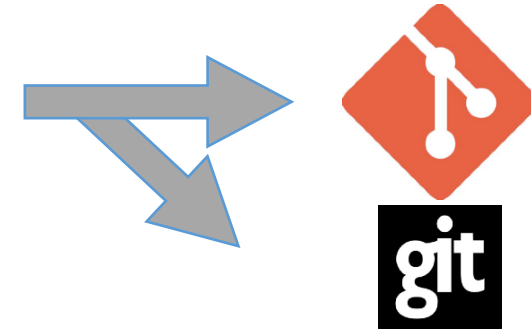


Stratégie de branchement de Git

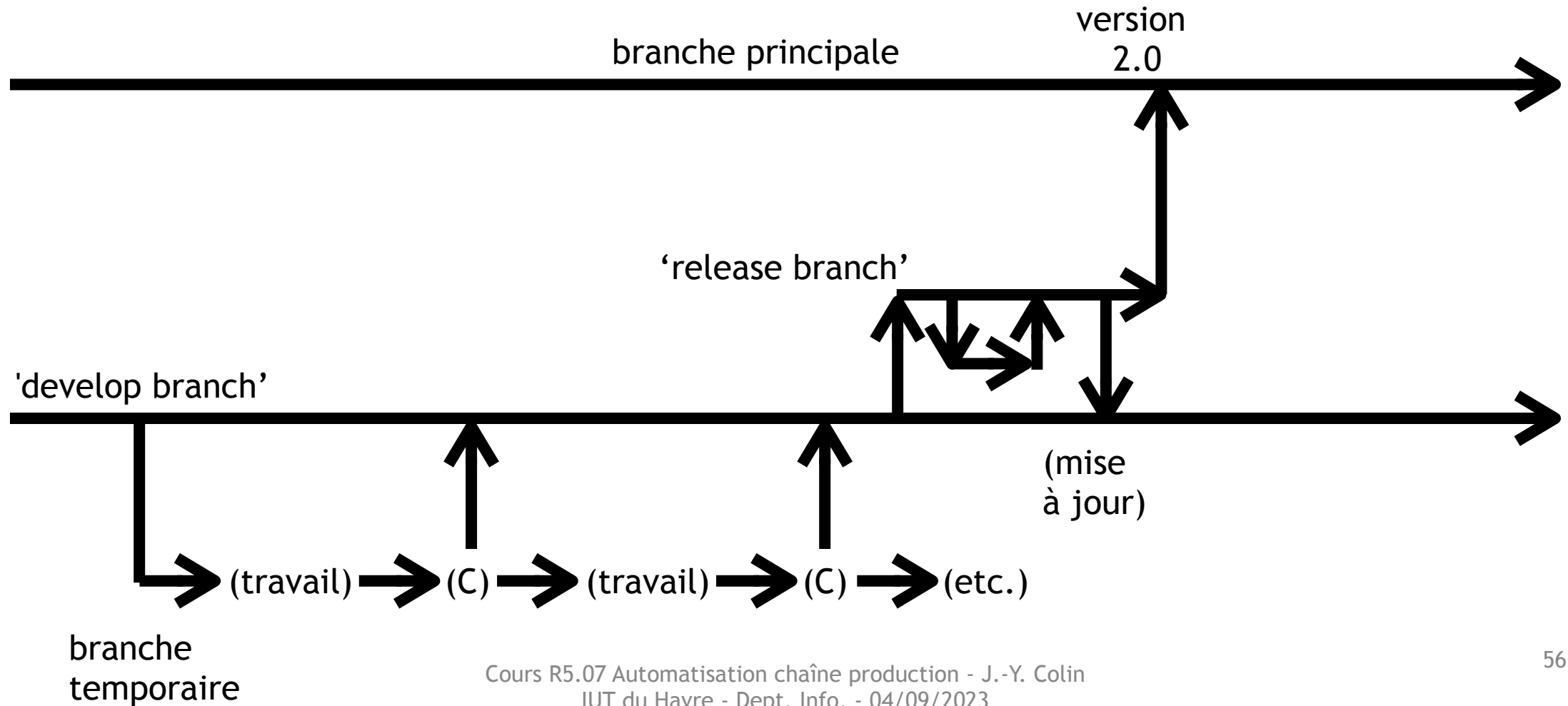
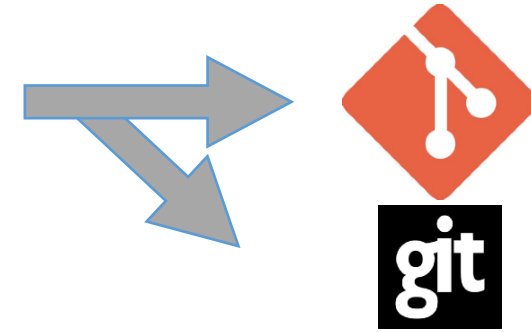


- lorsque la branche ‘release’ (ou une branche ‘release candidate’ si présente) est prête, le Git flow continue avec une fusion de cette branche vers la branche ‘main’. On obtient une version « majeure »
 - en cas de bug, les corrections (‘hotfix’) sont faites dans des sous-branches temporaires de la branche ‘main’ (pas de la branche ‘release’ ou de la branche ‘develop’).
 - puis les corrections dans la sous-branche ‘hotfix’ sont fusionnées dans la branche ‘main’ et dans la branche ‘develop’ pour les mettre à jour. On obtient une version « mineure »

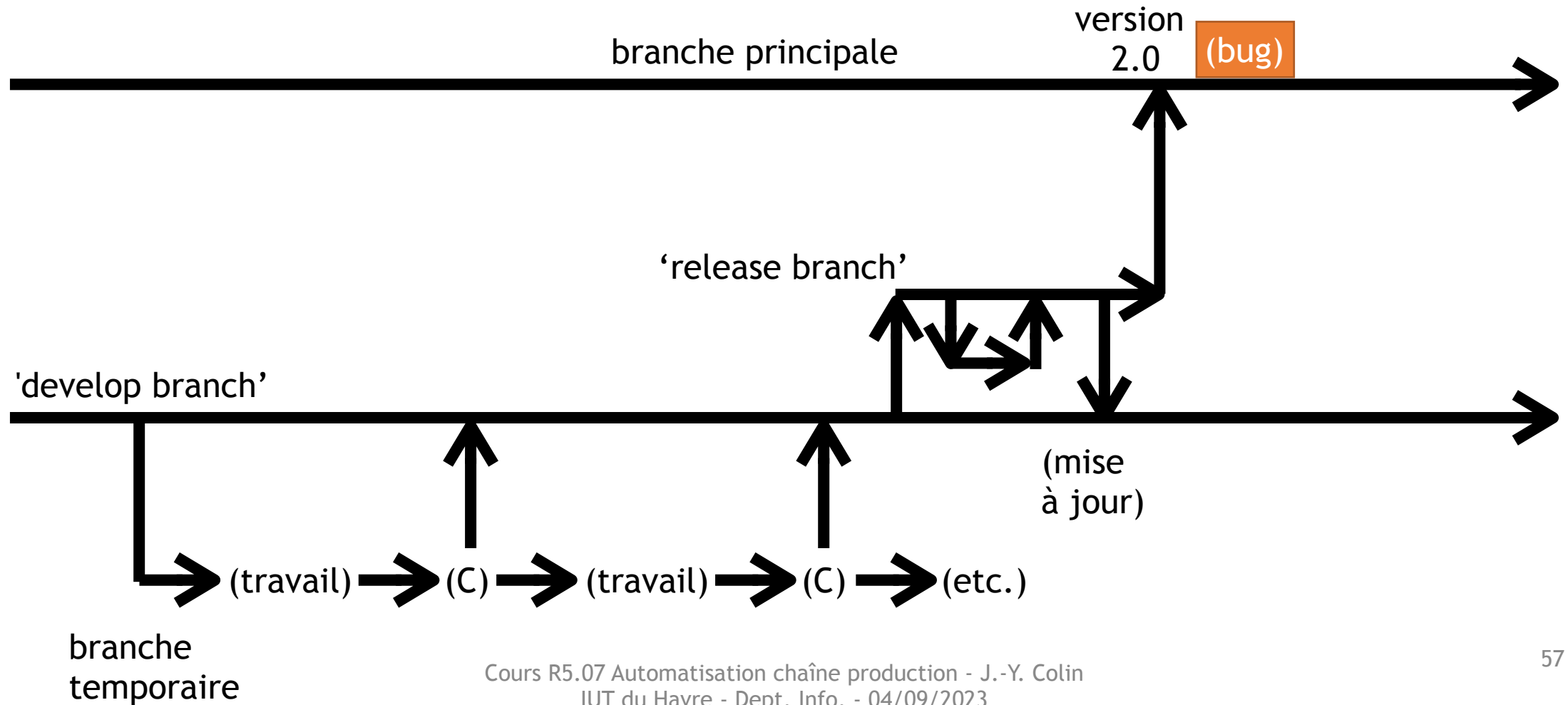
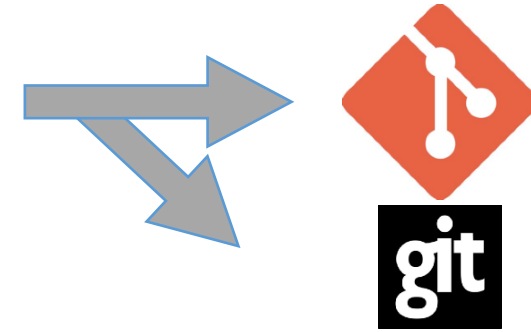
Stratégie de branchement de Git



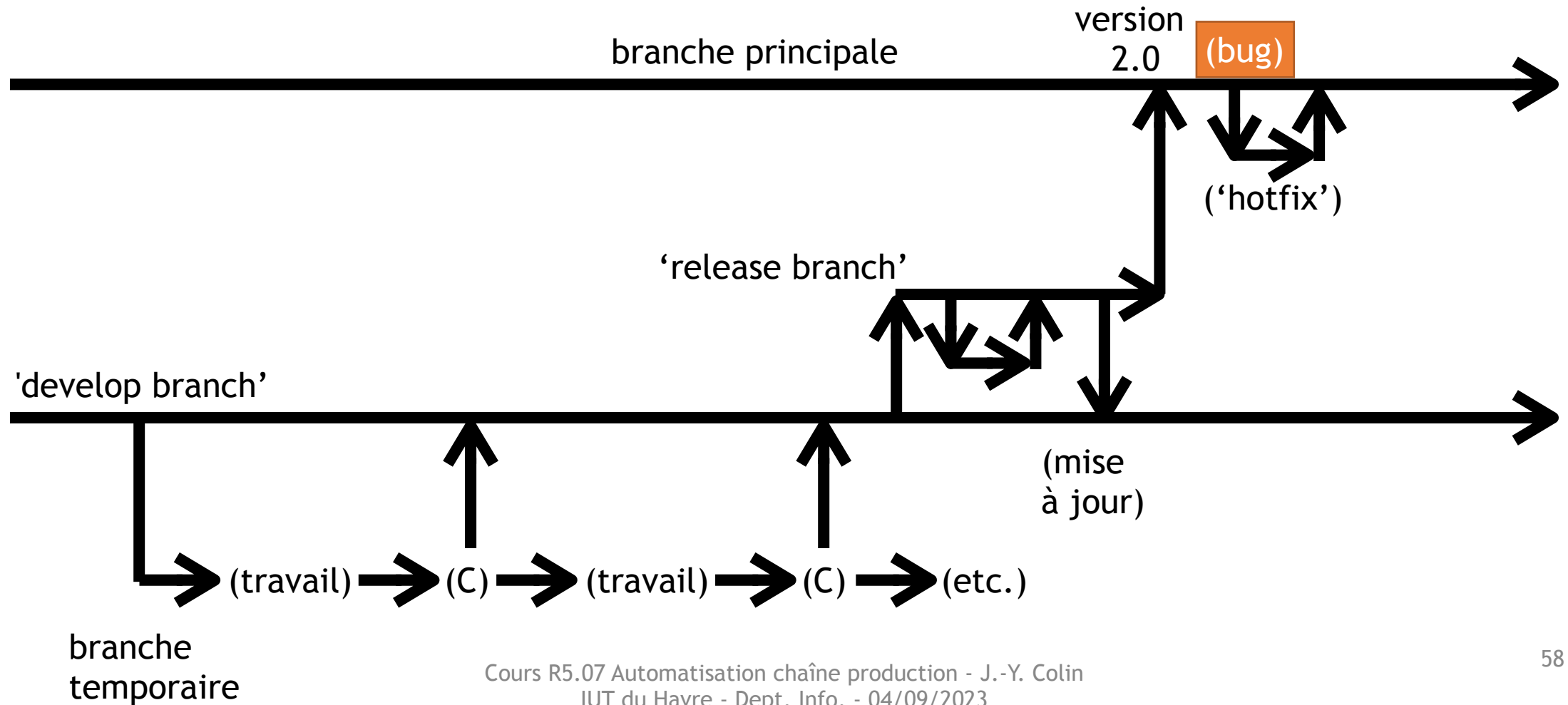
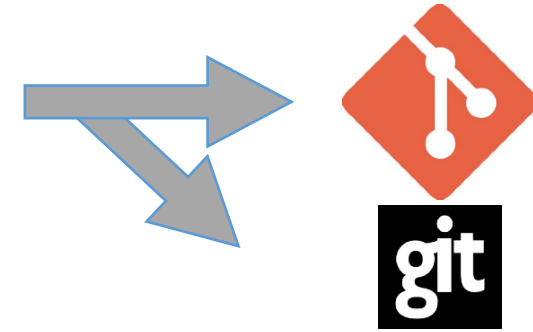
Stratégie de branchement de Git



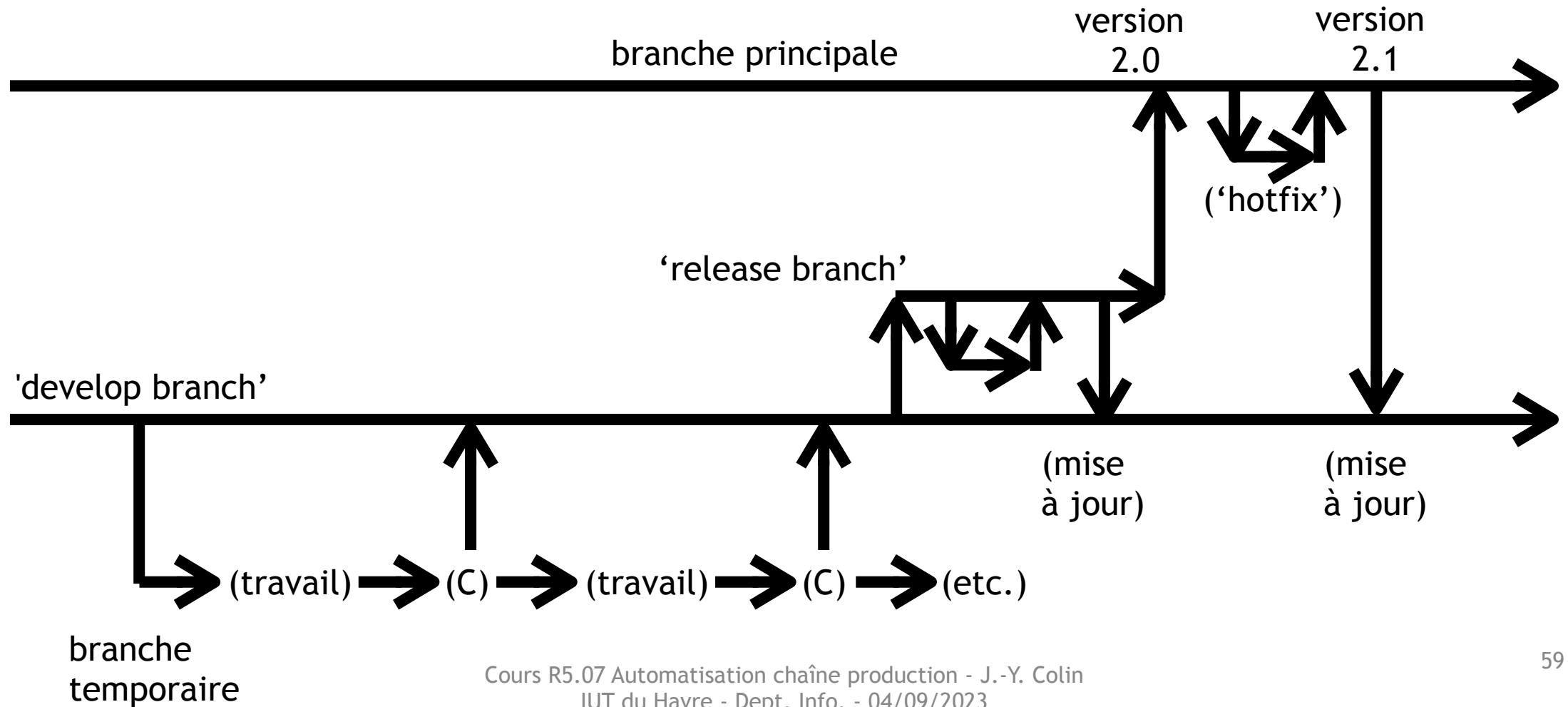
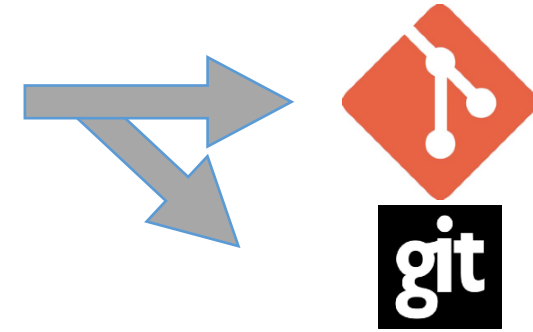
Stratégie de branchement de Git



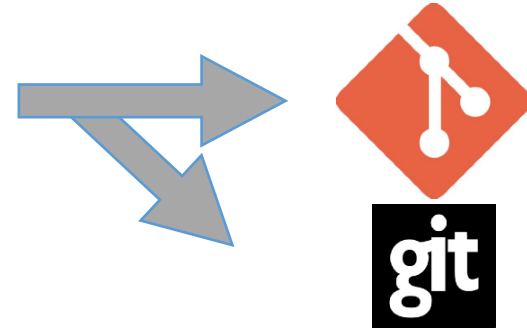
Stratégie de branchement de Git



Stratégie de branchement de Git



Stratégie de branchement de Git



- Comme on le voit, le workflow Git n'applique pas une politique 'upstream-first'